

# Chapter 2 - Finite Markov Decision Processes

**Hoda Zamani**

**PhD in Artificial Intelligence and Robotics**

**GitHub: [hodazamani.github.io](https://github.com/hodazamani)**

[Hoda Zamani - Google Scholar](#)

**Primary Textbook**

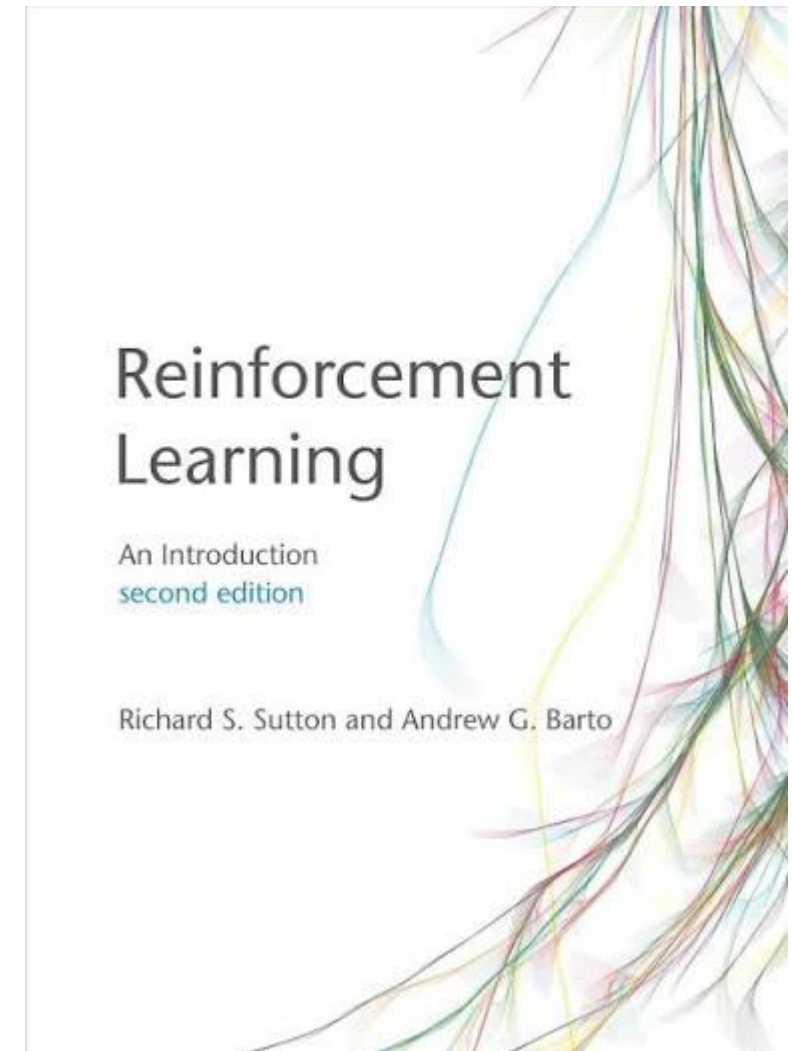
Reinforcement Learning: An Introduction, Second edition, in progress  
Richard S. Sutton and Andrew G. Barto, [Download from:  
SuttonBartoIPRLBook2ndEd.pdf](#)

**Recommended References**

- Szepesvári, C. (2010). Algorithms for Reinforcement Learning. Morgan & Claypool Publishers.
- Bertsekas, D. P. (2019). Reinforcement Learning and Optimal Control. Athena Scientific.
- Puterman, M. L. (2014). Markov Decision Processes: Discrete Stochastic Dynamic Programming. Wiley.

**Additional Course Materials**

- Instructor-provided lecture notes, slides, and supplementary reading materials.
- Programming assignments and practical exercises using Python and relevant machine learning libraries.
- Selected research papers and technical articles on reinforcement learning and decision-making.
- Computational demonstrations, datasets, and examples provided throughout the course.



You can find:

-  Course announcements & communication
-  Slides & weekly schedule
-  Textbook & reading materials
-  TA
-  Course policies

Here: <https://hodazamani.github.io/Courses/>

| Assessment Component                        | Weight      |
|---------------------------------------------|-------------|
| Weekly Assignments & Programming Exercises  | 20%         |
| Midterm Examination                         | 30%         |
| Reinforcement Learning Project/Presentation | 20%         |
| Final Examination                           | 30%         |
| <b>Total</b>                                | <b>100%</b> |

# CONTENTS

## Chapter 2- Finite Markov Decision Processes

- **The Agent–Environment Interface**
- **Goals and Rewards**
- **Returns**
- **Unified Notation for Episodic and Continuing Tasks**
- **The Markov Property**
- **Markov Decision Processes**
- **Value Functions**
- **Optimal Value Functions**
- **Optimality and Approximation**

## What Is Reinforcement Learning?

Reinforcement learning is the problem of learning to make decisions through interaction with an environment to achieve a goal.

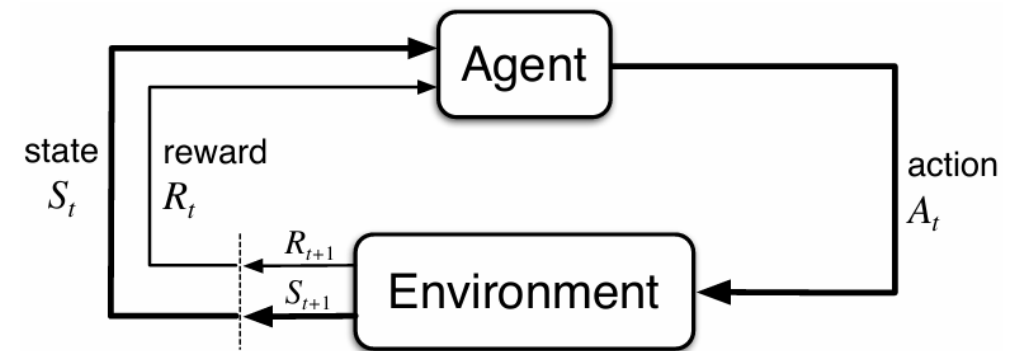
**Two fundamental components:**

•**Agent**

- Learner and decision-maker
- Selects actions
- Learns from experience

•**Environment**

- Everything outside the agent
- Responds to the agent's actions
- Produces new states and rewards



The agent observes a state, selects an action, and receives a reward and a new state from the environment.

**Simple process:**

**Agent  $\rightarrow$  Action  $\rightarrow$  Environment  $\rightarrow$  Reward + New State  $\rightarrow$  Agent**

Unlike supervised learning, where the learner is given correct answers, reinforcement learning involves interaction. The agent must decide what to do, observe what happens, and use the resulting experience to improve future decisions.

At each discrete time step:  $t = 0, 1, 2, 3, \dots$

1. Agent observes the current state:

- $S_t \in \mathcal{S}$

- $S_t$  = the state at time  $t$
- $\mathcal{S}$  = the set of all possible states

2. Agent selects an action:

- $A_t \in \mathcal{A}(S_t)$

where  $\mathcal{A}(S_t)$  is the set of actions available in state  $S_t$ .

3. Environment responds.

4. Agent receives a reward:

- $R_{t+1} \in \mathcal{R}$

5. Environment transitions to a new state:

- $S_{t+1}$

## Interaction

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1}$$

The important point is that the reward and next state occur after the action. The notation  $R_{t+1}$ , rather than  $R_t$ , emphasizes that the action  $A_t$  leads to the next reward and next state.

| Element                                                                                   | Meaning                                 |
|-------------------------------------------------------------------------------------------|-----------------------------------------|
| State $S_t$                                                                               | Current situation observed by the agent |
| Action $A_t$                                                                              | Decision made by the agent              |
| Reward $R_{t+1}$                                                                          | Numerical feedback from the environment |
| Environment                                                                               | Everything outside the agent            |
| Policy $\pi$                                                                              | Strategy used to select actions         |
| <b>Goal</b><br>The agent attempts to <b>maximize the total reward received over time.</b> |                                         |

States describe the information available to the agent. Actions represent choices. Rewards indicate how desirable the outcome was to the agent. The policy connects states to actions and is the mechanism through which behavior is represented.

A **policy** specifies how the agent chooses actions based on states.

It is represented by:

$$\pi_t(a|s)$$

where:

- $s$  = current state
- $a$  = possible action
- $\pi_t(a|s)$  = probability of selecting action  $a$  when in state  $s$

## Example

If a robot is deciding whether to:

- Search → 60%
- Wait → 25%
- Recharge → 15%

then these probabilities form part of its policy for that state.

**Key idea:**

Reinforcement learning methods modify the policy based on experience.

Reinforcement Learning (RL) is a flexible framework for solving sequential decision-making problems. Its flexibility comes from the fact that an RL agent does not need to be given a complete set of rules telling it exactly what to do in every situation. Instead, the agent **interacts with an environment, takes actions, receives rewards or penalties, and learns a strategy (policy) that improves its future decisions.**

1. Basic Reinforcement Learning framework

The main components are:

$Agent \leftrightarrow Environment$

At time  $t$ :

- 1. The agent observes the current state  $S_t$ .
- 2. It chooses an action  $A_t$ .
- 3. The environment responds.
- 4. The agent receives a reward  $R_{t+1}$ .
- 5. The environment moves to a new state  $S_{t+1}$ .

This can be represented as:

$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1} \rightarrow A_{t+1} \rightarrow \dots$

The objective is generally to maximize the **expected cumulative reward**.

2. Why is the RL framework flexible?

The RL framework is flexible because many different problems can be represented using the same basic structure:

$(S, A, P, R, \gamma)$

where:

- $S$  = state space
- $A$  = action space
- $P$  = transition dynamics
- $R$  = reward function
- $\gamma$  = discount factor

By changing these components, the same RL framework can represent very different applications.

For example:

| Application | State              | Action          | Reward              |
|-------------|--------------------|-----------------|---------------------|
| Robot       | Position, velocity | Move/turn       | Successful movement |
| Game        | Game configuration | Game move       | Win/loss            |
| Healthcare  | Patient condition  | Treatment       | Health outcome      |
| Finance     | Market information | Buy/sell/hold   | Financial return    |
| Traffic     | Traffic conditions | Signal decision | Reduced congestion  |

The **agent–environment boundary** is one of the most important concepts in **Reinforcement Learning (RL)**. It defines what belongs to the learning agent and what belongs to the outside world with which the agent interacts.

The basic idea is:

$\text{Agent} \longleftrightarrow \text{Environment}$

The agent **chooses actions**, while the environment **responds to those actions** by producing a new state and a reward.

An important point is that the agent–environment boundary is **not necessarily a physical boundary**.

It is primarily a **modeling boundary**.

For example, suppose a robot contains:

- sensors,
- motors,
- a processor,
- a planning algorithm.

Depending on the RL model, some of these components may be considered part of the agent and others may be treated as part of the environment.

Therefore:

Agent–environment boundary depends on how the problem is modeled.

Knowledge vs. Control is an important distinction in reinforcement learning (RL). The central idea is:

The agent may know how the environment behaves without being able to control what happens, and it may be able to control its actions without knowing exactly what will happen afterward.

In RL, separating what the agent knows from what the agent controls helps us understand the agent–environment boundary.

## 2. Knowledge

In RL, **knowledge** refers to information the agent has—or has learned—about the environment.

Suppose we have:

$$P(s'|s, a)$$

This is the transition model.

It tells us:

Given state  $s$  and action  $a$ , what is the probability that the next state will be  $s'$ ?

For example:

$$P(s_{\text{goal}}|s_{\text{current}}, a_{\text{right}}) = 0.8$$

means that if the agent is in  $s_{\text{current}}$  and takes  $a_{\text{right}}$ , there is an 80% probability of reaching the goal.

Knowing this transition probability is **knowledge**.

It does not mean the agent can force the goal to occur.

## 3. Control

**Control** means the ability to select or influence actions.

The agent chooses:

$$A_t$$

according to its policy:

$$\pi(a|s)$$

For example:

$$\pi(\text{right}|s) = 0.9$$

means that the agent chooses the action "right" with probability 0.9.

The agent controls this choice.

However, after choosing "right," it may not control the exact next state.

For example:

$$P(s_{\text{goal}}|s, \text{right}) = 0.8$$

$$P(s_{\text{wrong}}|s, \text{right}) = 0.2$$

The agent controls:

choose right

but does not necessarily control:

whether the result is goal or wrong

The RL framework abstracts a complicated learning system into three signals:

## 1. State

"What situation am I in?"

## 2. Action

"What should I do?"

## 3. Reward

"How good was the outcome?"

Basic relationship

$$\text{State} \rightarrow \text{Action} \rightarrow \text{Reward} + \text{New State}$$

Main idea:

RL focuses on learning **how to behave**, once states and actions have been represented.

**Representation matters** means that how we represent the state, action, reward, and knowledge of an RL problem can strongly affect what the agent can learn and how efficiently it learns.

Suppose two agents receive exactly the same environment.

Agent A receives a useful representation:

$$S = (\text{distance to goal, direction to goal})$$

while Agent B receives irrelevant information:

$$S = (\text{wall color, floor texture, camera noise, } \dots).$$

Both technically have information about the environment.

But Agent A's representation makes the important structure easier to discover.

Therefore:

Good representation  $\rightarrow$  easier learning

while:

Poor representation  $\rightarrow$  harder learning

## Task

Control a bioreactor to maximize production of a useful chemical.

## Agent controls:

- Target temperature
- Target stirring rate

## State includes:

- Thermocouple readings
- Other sensor measurements
- Filtered/delayed measurements
- Ingredients in the vat
- Target chemical information

## Reward

- Rate at which useful chemical is produced

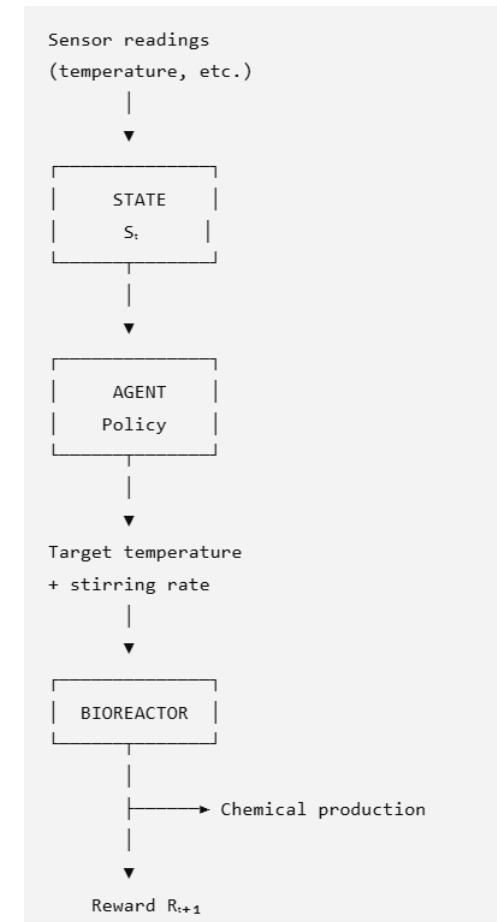
## Important observation

The state may be a **vector/list of values**.

The action may also be a **vector**:

$$A_t = [\text{target temperature, target stirring rate}]$$

But the reward is always a **single numerical value**.



The agent uses sensor information to select temperature and stirring targets, while chemical production provides feedback through the reward.

Example 1-- Bioreactor

| RL Component | Bioreactor                               |
|--------------|------------------------------------------|
| Agent        | RL control system                        |
| Environment  | Bioreactor + lower-level control systems |
| State        | Sensor readings + symbolic information   |
| Action       | Target temperature + stirring rate       |
| Reward       | Chemical production rate                 |
| Goal         | Increase useful chemical production      |

Example interaction

Sensor readings → Choose temperature/stirring → Chemical production → Reward

### Objective

Learn robot movements that are:

- Fast
- Smooth
- Successful

### Actions

Direct motor control, such as:

$$A_t = [\text{motor voltages at each joint}]$$

### States

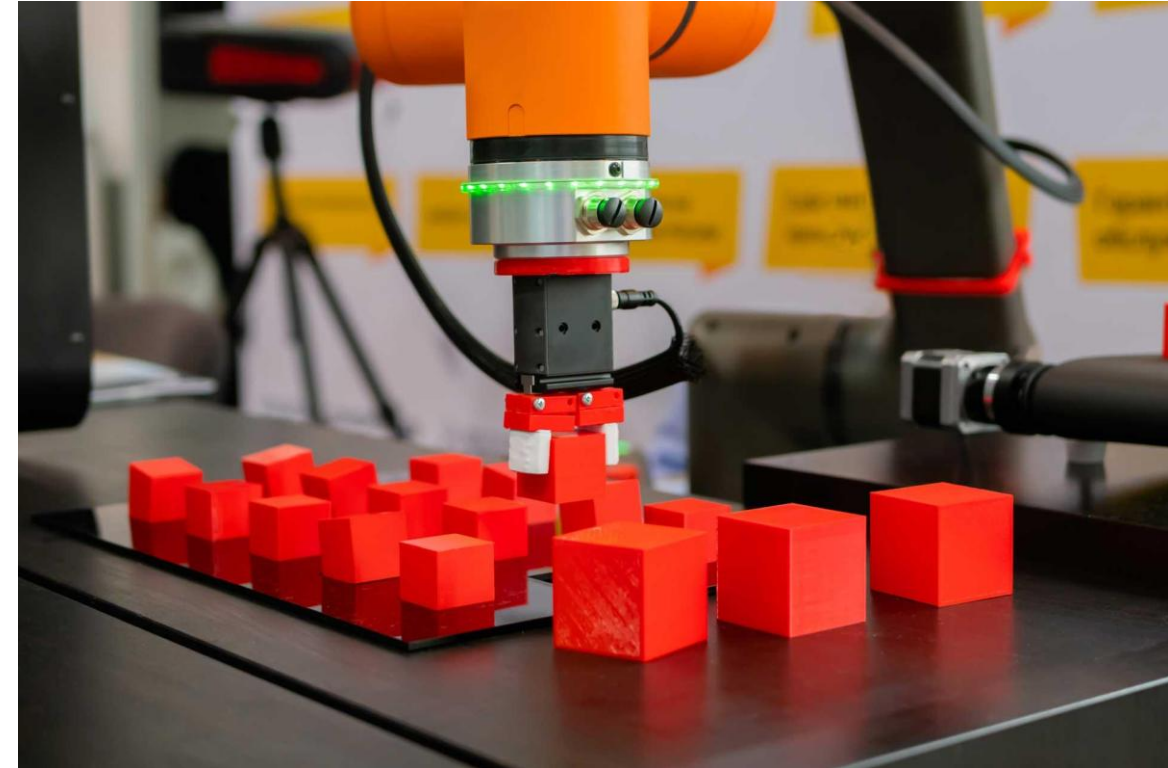
- Joint angles
- Joint velocities
- Current mechanical configuration

### Rewards

- +1 for successfully picking and placing an object
- Small negative reward for undesirable movement/jerkiness

Goal:

Successful task + Smooth movement



The agent uses joint information to control the motors and learns movements that successfully manipulate objects while reducing undesirable motion.

The robot needs to achieve the task **and** move smoothly.

## Reward structure

Successful pick-and-place:

$$R = +1$$

Jerky movement:

$$R < 0$$

## Why use negative rewards?

They discourage undesirable behavior.

## General principle

The reward function communicates:

What behavior should be encouraged or discouraged?

Reward design is extremely important. If we only reward successful placement, the robot might discover movements that accomplish the task but are unnecessarily aggressive or jerky. Additional penalties can encourage smoother behavior.

### Task

A mobile robot collects empty soda cans.

The robot has:

- Can-detection sensors
- Arm and gripper
- Battery
- Navigation system

**RL agent makes high-level decisions:**

1. Search for a can
2. Wait for someone to bring a can
3. Return home to recharge

### State

Primarily determined by:

Battery charge level



The RL agent makes high-level decisions, while other robot components execute those decisions. This illustrates that the agent need not be identical to the entire physical robot.

## Recycling Robot — Rewards

Rewards might be:

- 0 most of the time
- **Positive** when a can is successfully collected
- **Large negative reward** if the battery becomes completely depleted

### Example

$$R = \begin{cases} +R_{\text{can}} & \text{can successfully collected} \\ 0 & \text{normal operation} \\ -R_{\text{battery}} & \text{battery completely depleted} \end{cases}$$

### Decision problem

The robot must balance:

Collecting cans

vs.

Preserving enough battery to return and recharge

## The Entire Robot Is Not Necessarily the Agent In the recycling example:

### Agent

High-level decision-making system

### Environment

- Robot's navigation system
- Arm and gripper
- Sensors
- Battery
- External office environment
- Other control systems

### Key idea

The agent can be only one decision-making component within a larger system.

This demonstrates why the agent–environment boundary is **conceptual and task-dependent**.

Three RL Applications

| Feature      | Bioreactor             | Pick-and-Place          | Recycling Robot                  |
|--------------|------------------------|-------------------------|----------------------------------|
| Goal         | Produce chemicals      | Pick/place objects      | Collect cans                     |
| State        | Sensors + symbols      | Joint angles/velocities | Battery state                    |
| Action       | Temperature + stirring | Motor voltages          | Search / wait / recharge         |
| Reward       | Production rate        | Success + smoothness    | Can collection / battery failure |
| Action level | Medium/high            | Low-level               | High-level                       |
| Agent        | Control system         | Robot controller        | High-level decision system       |

## 1. RL is highly flexible

It can control:

- Industrial processes
- Robots
- Autonomous systems

## 2. States can have different forms

- Numeric vectors
- Sensor readings
- Symbolic information
- Internal conditions

## 3. Actions can be different levels

- Direct physical control
- Intermediate commands
- High-level decisions

## 4. Rewards define the objective

They tell the agent what outcomes are desirable.

## 5. The agent can be a component

The agent does not have to be the entire physical system.

## Goals and Rewards

In Reinforcement Learning, these three concepts explain what the agent wants, what it receives, and how it evaluates the future.

### 1. Goal

The **goal** is what the agent is trying to achieve.

Example:

Start → State 1 → State 2 → Goal



For a robot, the goal could be **reaching a destination**.

### 2. Reward $R_t$

A **reward** is a numerical signal received after an action.

Example:

Agent  $\xrightarrow[\text{+10}]{\text{Move}}$  Goal

- Reaching the goal:  $R = +10$
- Normal movement:  $R = 0$
- Bad action:  $R = -1$

The reward tells the agent **how good or bad** the immediate outcome was.

### 3. Return $G_t$

The **return** is the total amount of reward the agent expects to receive from time  $t$  onward.

The basic formula is:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

With discounting:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where:

- $G_t$  = return
- $R_{t+1}, R_{t+2}, \dots$  = future rewards
- $\gamma$  = discount factor,  $0 \leq \gamma \leq 1$

## Goals and Rewards

### Numerical Example

Suppose an agent follows this path:

Start → State 1 → State 2 → Goal  
-1            -1            +10

Let:

$$\gamma = 0.9$$

Then:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$$G_0 = -1 + 0.9(-1) + 0.9^2(10)$$

$$G_0 = -1 - 0.9 + 8.1$$

$$G_0 = 6.2$$

## Definition

All goals and purposes can be understood as the maximization of the expected value of the cumulative sum of a received scalar reward signal.

In simpler words:

Goal  $\approx$  Maximize expected cumulative reward

## Three important concepts

- **Reward**  $\rightarrow$  numerical feedback
- **Cumulative**  $\rightarrow$  considers multiple rewards over time
- **Expected**  $\rightarrow$  considers uncertainty and possible future outcomes

This is one of the most important ideas in reinforcement learning.  
Instead of explicitly programming an agent with a complete description of what we want it to do, we define a reward signal that represents the desired outcome.  
The agent then learns a policy that attempts to maximize the expected cumulative reward.

## Reward Signals Can Represent Many Goals

| Task                                                                                                  | Possible Reward                          |
|-------------------------------------------------------------------------------------------------------|------------------------------------------|
| Robot walking                                                                                         | Reward proportional to forward movement  |
| Maze escape                                                                                           | -1 for every step before escape          |
| Recycling                                                                                             | +1 for each can collected                |
| Avoiding collisions                                                                                   | Negative reward for bumping into objects |
| Chess/checkers                                                                                        | +1 win, -1 loss, 0 draw                  |
| <b>Key idea</b><br>Different real-world objectives can be translated into <b>numerical feedback</b> . |                                          |

The reward mechanism appears simple, but it is surprisingly flexible. For example, in a maze, giving the robot -1 at every time step encourages it to reach the exit quickly. It does not need a separate rule saying "be fast." The reward structure creates that incentive.

The agent learns:

What behavior maximizes reward?

Therefore:

**If we want the agent to achieve goal  $G$**

we must design rewards such that:

Maximizing reward  $\Rightarrow$  Achieving  $G$

**Critical principle**

The reward tells the agent **WHAT** to achieve, not **HOW** to achieve it.

## Chess example

Suppose the real goal is:

Win the game

A poor reward design might give rewards for:

- Capturing opponent pieces
- Controlling the center
- Moving pieces toward the opponent
- Achieving particular intermediate positions

## Problem

The agent may learn to maximize these subgoals **without actually winning**.

## Better formulation

+1 = Win

-1 = Lose

0 = Draw / nonterminal position

The reward is considered part of the **environment**, not the agent.

## Why?

The agent should have:

- Control over its actions
- **Imperfect control over its rewards**

The agent should not be able to simply declare:

"I achieved the goal, therefore give me the reward."

## Agent–environment boundary

|                                        |
|----------------------------------------|
| Agent's control   Environment + Reward |
|----------------------------------------|

Consider a robot whose goal is related to:

- Battery/energy level
- Position of its limbs
- Internal temperature

These variables may physically exist **inside the robot**.

However, from the RL perspective, they can still be considered part of the **environment**.

**Why?**

Because the learning agent cannot arbitrarily control them.

**Key principle**

Physical location  $\neq$  Agent boundary

Instead:

Control boundary determines agent/environment

The formal RL framework treats the reward source as external to the learning agent.

However, an agent can internally construct:

- Internal rewards
- Internal reward sequences
- Additional learning signals

## Important distinction

Formal reward:

Environment  $\rightarrow$  Agent

Internal learning mechanisms:

Agent's internal processes

So far:

The agent wants to maximize cumulative reward.

But what exactly does **cumulative reward** mean?

Suppose after time  $t$ , the rewards are:

$$R_{t+1}, R_{t+2}, R_{t+3}, \dots$$

We need a formal definition of what the agent is trying to maximize.

**This quantity is called the:**

$$\boxed{\text{Return } G_t}$$

**General idea**

$$\boxed{\text{Return} = \text{some function of future rewards}}$$

For an episodic task:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

where:

- $G_t$  = return at time  $t$
- $R_{t+k}$  = future reward
- $T$  = final time step

## Example

Suppose the future rewards are:

1, 2, 3, 4

Then:

$$G_t = 1 + 2 + 3 + 4 = 10$$

This is the simplest definition of return: add all the rewards from the current point until the end of the episode. It works particularly well when there is a natural ending point.

An **episode** is a sequence of interactions that naturally ends.

## Examples

- Playing a chess game
- Playing a checkers game
- Escaping a maze
- Completing a robot task
- One attempt at balancing a pole

An **episodic task** is a Reinforcement Learning problem that has a **clear beginning** and a **terminal ending**.

-  Playing a game → ends when the game is over
-  Robot reaching a destination → ends when it reaches the goal
-  Golf → ends when the ball reaches the hole
-  Solving a puzzle → ends when the puzzle is solved

## Structure

Start → Interactions → Terminal State

Then the environment is reset.

## Terminology

- Nonterminal states:  $S$
- All states including terminal:  $S^+$

A **continuing task** is a Reinforcement Learning problem that **does not have a terminal state**. The agent continues interacting with the environment indefinitely.

There is **no Goal/Terminal State**.

## Numerical Example

Suppose the robot receives:

|         |    |    |    |    |    |       |
|---------|----|----|----|----|----|-------|
| Time:   | 0  | 1  | 2  | 3  | 4  |       |
| Reward: | +2 | +1 | +3 | +1 | +2 | → ... |

With:

$$\gamma = 0.9$$

The return is:

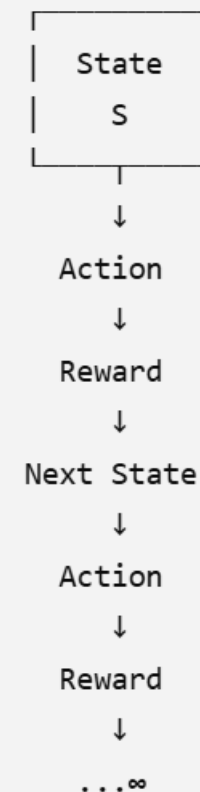
$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

For example:

$$G_0 = 2 + 0.9(1) + 0.9^2(3) + 0.9^3(1) + \dots$$

The rewards can continue **forever**, so the discount factor helps keep the return finite.

## Example: Robot in a Factory



## Key Difference

### Episodic Task

Has a terminal state

Episode eventually ends

Example: Game

$V(\text{terminal}) = 0$

### Continuing Task

No terminal state

Continues indefinitely

Example: Factory robot

No terminal state

## Discounted Return

The **discounted return** is the total future reward where **rewards received later are given less weight**.

### Formula

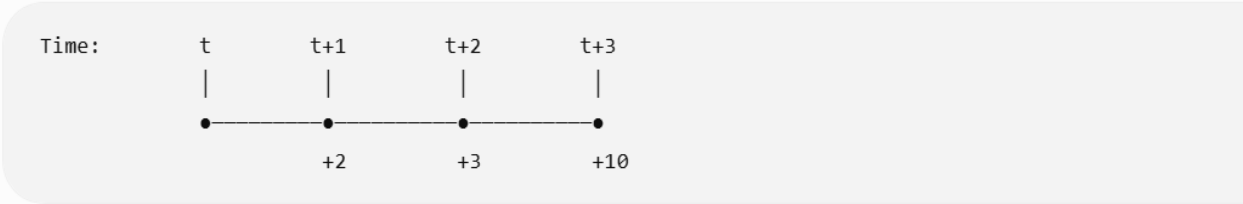
$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots$$

### Meaning of Variables

- $G_t$  = **Return** from time  $t$
- $R_{t+1}$  = reward received **one step later**
- $R_{t+2}$  = reward received **two steps later**
- $\gamma$  = **discount factor**
- $0 \leq \gamma \leq 1$

# Example - Discounted Return

Suppose an agent receives these rewards:



We assume the **discount factor** is:

$$\gamma = 0.9$$

## 1. Discounted Return Formula

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3}$$

The discount factor  $\gamma = 0.9$  is **given as an assumption**. It is not calculated from the rewards.

Each reward is multiplied by a different power of  $\gamma$ .

### First reward

The first future reward is one step away:

$$\gamma^0 = 0.9^0 = 1$$

So its discount factor is:

$$1.00$$

### Second reward

The second future reward is two steps away:

$$\gamma^1 = 0.9^1 = 0.9$$

So:

$$0.90$$

### Third reward

The third future reward is three steps away:

$$\begin{aligned} \gamma^2 &= 0.9^2 \\ &= 0.9 \times 0.9 = 0.81 \end{aligned}$$

So:

$$0.81$$

Example - Discounted Return

4. Final Calculation

$$G_t = (2)(1.00) + (3)(0.90) + (10)(0.81)$$

$$G_t = 2 + 2.7 + 8.1$$

$G_t = 12.8$

5. The Important Pattern

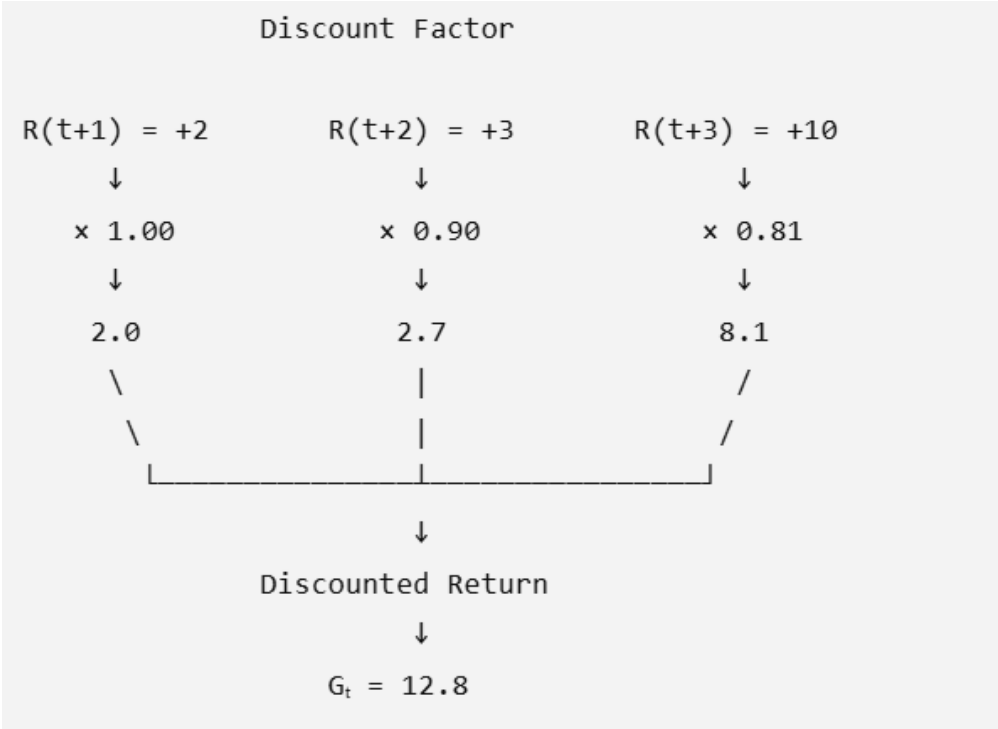
The discount factors follow this pattern:

$1, \gamma, \gamma^2, \gamma^3, \dots$

For  $\gamma = 0.9$ :

$1.00, \quad 0.90, \quad 0.81, \quad 0.729, \dots$

So, the farther a reward is in the future, the smaller its discount factor becomes.



This concept explains **how much an agent cares about future rewards**.

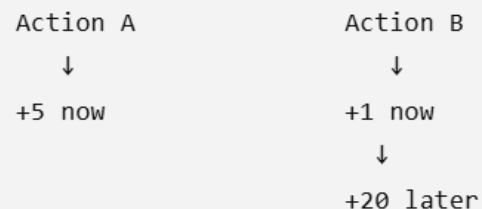
## 1. Myopic Agent 🧐

A **myopic agent** mainly cares about the **immediate reward**.

This happens when:

$$\gamma \approx 0$$

For example:



A myopic agent focuses mainly on:

$$+5$$

and may not care much about the +20 later.

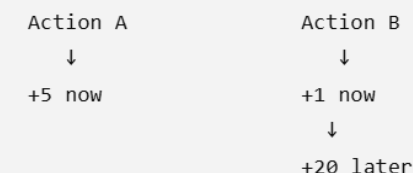
## 2. Farsighted Agent 🔭

A **farsighted agent** considers **future rewards** as well as immediate rewards.

This happens when:

$$\gamma \approx 1$$

Using the same example:



With:

$$\gamma = 0.9$$

For Action A:

$$G_A = 5$$

For Action B:

$$G_B = 1 + 0.9(20)$$

$$G_B = 1 + 18 = 19$$

So the **future reward has a large effect** on the return.

A myopic agent asks: *"What reward can I get now?"*

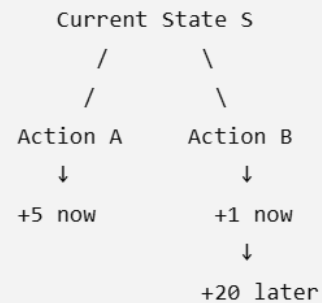
A farsighted agent asks: *"What rewards can I get now and in the future?"*

In Reinforcement Learning, an agent should **not** choose an action only by looking at the immediate reward.

The reason is that an action can have a **small** reward now but **much better** rewards later.

## Example

Suppose an agent has two choices:



If we only look at the immediate reward:

$$A = +5$$

$$B = +1$$

The agent might choose **A**.

But suppose:

$$\gamma = 0.9$$

For Action B:

$$G_B = 1 + 0.9(20)$$

$$G_B = 1 + 18 = 19$$

So:

$$G_B = 19$$

while:

$$G_A = 5$$

**Why?**

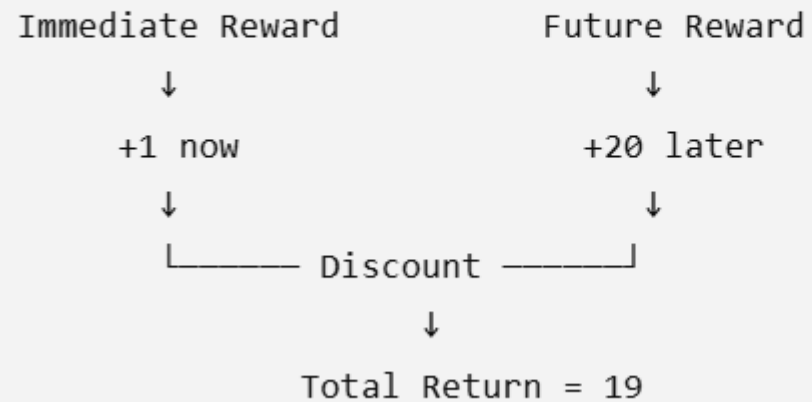
Because the **total return** considers future rewards:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

In Reinforcement Learning, an agent should **not** choose an action only by looking at the immediate reward.

The reason is that an action can have a **small** reward now but **much better** rewards later.

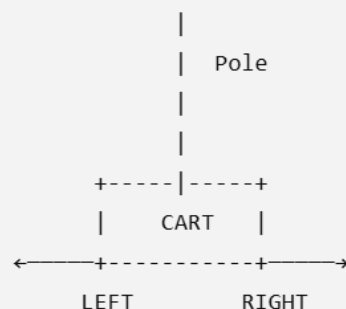
Therefore:



Immediate reward tells us what we get now, but the return tells us how good the consequences of an action are over time.

**Pole balancing** is a classic Reinforcement Learning example. The agent must learn how to keep a pole upright by moving a cart.

## 1. Environment



The cart can move left or right.

## 2. Goal

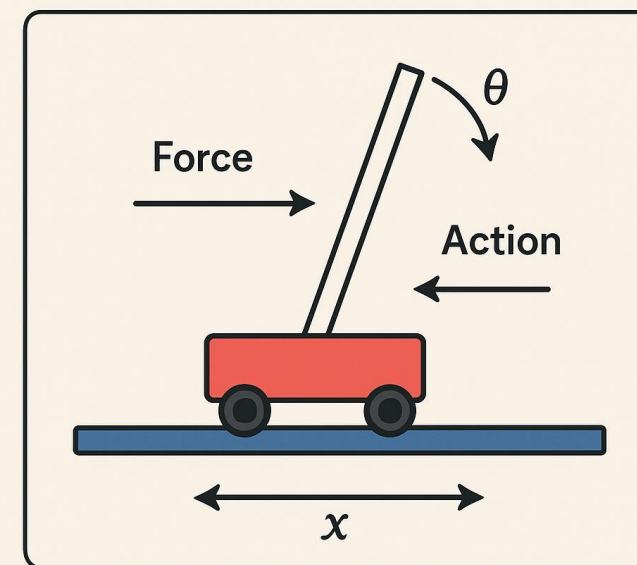
The goal is to:

Keep the pole upright for as long as possible

The episode ends when:

- the pole falls too far, or
- the cart moves outside the allowed area.

## CartPole in OpenAI Gym



**Pole balancing** is a classic Reinforcement Learning example. The agent must learn how to keep a pole upright by moving a cart.

## 3. States

The state can include:

$$s = (x, \dot{x}, \theta, \dot{\theta})$$

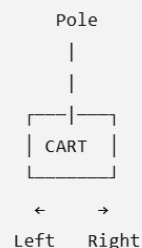
where:

- $x$  = cart position
- $\dot{x}$  = cart velocity
- $\theta$  = pole angle
- $\dot{\theta}$  = pole angular velocity

## 4. Actions

The agent has two actions:

$$A = \{\text{Left}, \text{Right}\}$$



## 5. Rewards

A simple reward system is:

$$R = +1$$

for every time step that the pole remains balanced.

If the pole falls:

$$R = 0$$

and the episode terminates.

**Pole balancing** is a classic Reinforcement Learning example. The agent must learn how to keep a pole upright by moving a cart.

## 6. Numerical Example

Suppose the agent keeps the pole balanced for 5 steps:

|         |    |    |    |    |    |
|---------|----|----|----|----|----|
| Step:   | 1  | 2  | 3  | 4  | 5  |
| Reward: | +1 | +1 | +1 | +1 | +1 |

Without discounting:

$$G_0 = 1 + 1 + 1 + 1 + 1$$

$$G_0 = 5$$

With:

$$\gamma = 0.9$$

the discounted return is:

$$\begin{aligned} G_0 &= 1 + 0.9(1) + 0.9^2(1) + 0.9^3(1) + 0.9^4(1) \\ &= 1 + 0.9 + 0.81 + 0.729 + 0.6561 \end{aligned}$$

$$G_0 = 4.0951$$

### 🧠 Main Idea

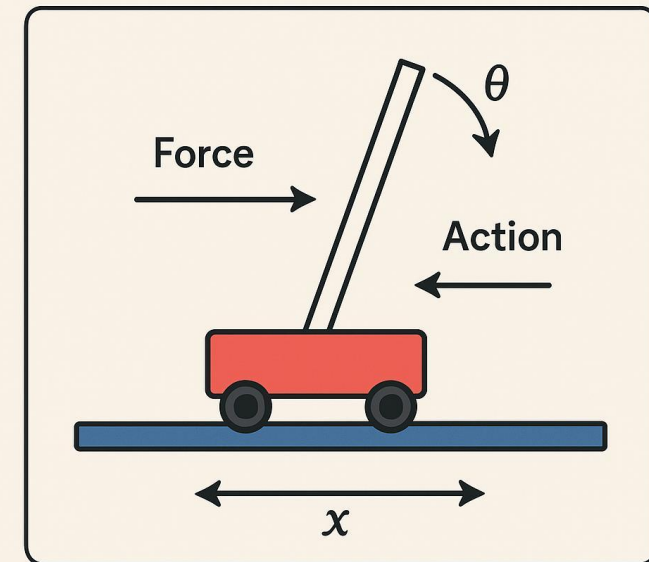
The agent learns:

“If I move the cart in the correct direction, I receive rewards and keep the pole balanced longer.”

Therefore, the agent learns a policy:

$$\pi(s) \rightarrow \text{Left or Right}$$

## CartPole in OpenAI Gym

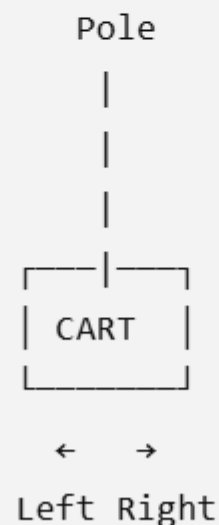


## Pole-Balancing as an Episodic Task

Pole balancing is an **episodic task** because each trial has a **clear beginning** and a **terminal condition**.

### 1. Basic Idea

The agent controls a cart and tries to keep the pole upright.



# Pole-Balancing as an Episodic Task

## 2. Episode

An episode starts when the cart and pole are initialized:



The episode ends when:

- the **pole falls too far**, or
- the **cart moves outside the allowed position**.

## 3. Example with Rewards

Suppose the agent successfully balances the pole for 5 time steps:

|         |    |    |    |    |    |            |
|---------|----|----|----|----|----|------------|
| Time:   | 0  | 1  | 2  | 3  | 4  | 5          |
|         | ↓  | ↓  | ↓  | ↓  | ↓  | ↓          |
| Reward: | +1 | +1 | +1 | +1 | +1 | 0          |
|         |    |    |    |    |    | ↑          |
|         |    |    |    |    |    | Pole falls |
|         |    |    |    |    |    | TERMINAL   |

Therefore, the episode contains **5 successful steps**.

## 4. Return

If:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$\gamma = 0.9$$

then:

$$\begin{aligned} G_0 &= 1 + 0.9(1) + 0.9^2(1) + 0.9^3(1) + 0.9^4(1) \\ &= 1 + 0.9 + 0.81 + 0.729 + 0.6561 \end{aligned}$$

$G_0 = 4.0951$

## 5. Why Is It Episodic?

Because the task has:

Start → Sequence of actions → Terminal state

After the pole falls, the **episode ends**, and a new episode can begin.

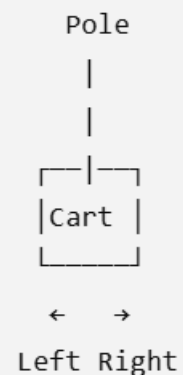
Pole balancing is episodic because the agent's interaction has a definite ending: the episode terminates when the pole falls or the cart leaves the allowed region.

## Pole-Balancing as a Continuing Task

Pole balancing can also be modeled as a **continuing task** if we remove the idea of a terminal state.

### Basic Idea

The agent continuously controls the cart to keep the pole upright:



There is **no terminal state**:

```

State  $s_0$ 
  ↓
Action
  ↓
State  $s_1$ 
  ↓
Action
  ↓
State  $s_2$ 
  ↓
Action
  ↓
State  $s_3$ 
  ↓
...
  ↓
 $\infty$ 
    
```

## Example with Rewards

Suppose the robot receives  $+1$  whenever the pole remains balanced:

|         |    |    |    |    |    |    |    |     |
|---------|----|----|----|----|----|----|----|-----|
| Time:   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | ... |
|         | ↓  | ↓  | ↓  | ↓  | ↓  | ↓  | ↓  |     |
| Reward: | +1 | +1 | +1 | +1 | +1 | +1 | +1 | ... |

Because the task continues indefinitely:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Assume:

$$\gamma = 0.9$$

Then:

$$G_t = 1 + 0.9 + 0.9^2 + 0.9^3 + \dots$$

This is an infinite geometric series:

$$G_t = \frac{1}{1 - \gamma}$$

$$G_t = \frac{1}{1 - 0.9}$$

$$\boxed{G_t = 10}$$

## Episodic vs. Continuing

### Episodic

Has a terminal state

Episode ends

Pole falls → END

New episode starts

### Continuing

No terminal state

Interaction continues

No END

Same task continues

## Main Idea

Continuing Pole-Balancing = Balance the pole indefinitely

## Two Formulations, Same Objective

The same Reinforcement Learning problem can sometimes be formulated in **two ways**:

1. Episodic formulation
2. Continuing formulation

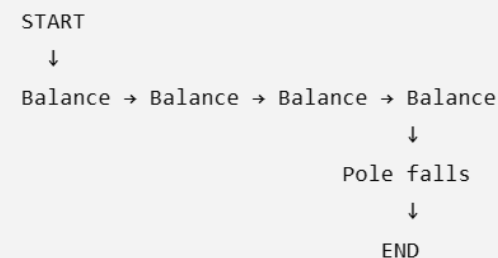
Both can have the **same objective**: keep the pole balanced for as long as possible.

### Pole-Balancing Example



## 1. Episodic Formulation

The episode **ends** when the pole falls.



For example:

$$R = +1$$

for every successful time step.

If the pole stays balanced for 5 steps:

$$G_0 = 1 + 1 + 1 + 1 + 1$$

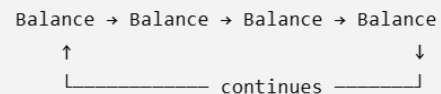
$$G_0 = 5$$

The objective is:

Maximize the length of the episode

## 2. Continuing Formulation

There is **no terminal state**.



The agent receives:

$$R = +1$$

continuously while balancing.

With:

$$\gamma = 0.9$$

the return is:

$$G_t = 1 + 0.9 + 0.9^2 + 0.9^3 + \dots$$

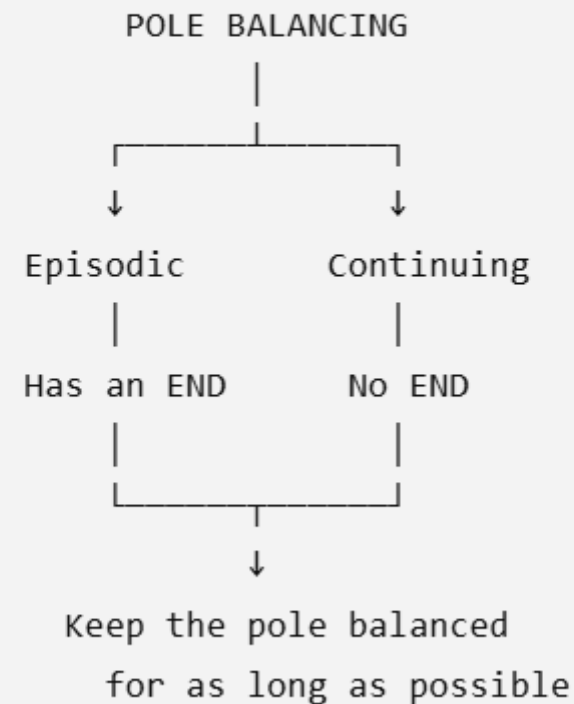
$$G_t = \frac{1}{1 - 0.9}$$

$$G_t = 10$$

The objective is:

Maximize long-term reward

## Same Objective



The process is:

## 1. Goal

What do we want the agent to accomplish?

↓

## 2. Reward

How can we communicate that goal numerically?

↓

## 3. Return

How do we combine rewards over time?

↓

## 4. Policy

How should the agent choose actions to maximize expected return?

**Overall relationship**

|                                 |
|---------------------------------|
| Goal → Reward → Return → Policy |
|---------------------------------|

## Two types of RL tasks

### Episodic tasks

- Agent–environment interaction naturally divides into separate **episodes**.
- Each episode contains a finite sequence of time steps.
- Each episode ends in a **terminal state**.
- Examples: chess game, poker hand, one trial of pole balancing.

### Continuing tasks

- No natural terminal point.
- Interaction continues indefinitely.
- Examples: continuous robot control, industrial process control.

For episodic tasks, technically we should identify the episode:

$$S_{t,i}, A_{t,i}, R_{t,i}, \pi_{t,i}, T_i$$

where:

- $t$  = time step
- $i$  = episode number
- $T_i$  = terminal time of episode  $i$

## In practice

The episode number is usually omitted:

$$S_t, A_t, R_t, \pi_t$$

because we usually discuss:

- one particular episode, or
- something true for all episodes.

## Main idea

The same notation can be used without repeatedly writing the episode number.

The idea of a **terminal state as an absorbing state** is an important concept in **Markov chains**, especially in models of disease progression, survival, reliability, finance, decision processes, and other systems where a process can eventually reach an endpoint.

## 1. What is a state?

A **state** describes the condition of a system at a particular time.

Let

$$X_t$$

denote the state of the system at time  $t$ .

For example, in a disease-progression model we could have:

$$S_1 = \text{Healthy}$$

$$S_2 = \text{Diseased}$$

$$S_3 = \text{Recovered}$$

$$S_4 = \text{Dead}$$

At any time  $t$ , the individual is assumed to be in one of these states.

## 2. What is a transition?

A **transition** means moving from one state to another.

For example,

$$S_1 \rightarrow S_2$$

means that the system moves from state  $S_1$  to state  $S_2$ .

Mathematically, the probability of this transition is written as

$$p_{ij} = P(X_{t+1} = j \mid X_t = i).$$

Here:

- $X_t$  = current state
- $X_{t+1}$  = next state
- $i$  = current-state index
- $j$  = next-state index
- $p_{ij}$  = probability of moving from state  $i$  to state  $j$

For example,

$$p_{12} = 0.30$$

means there is a 30% probability of moving from state 1 to state 2 during the next time interval.

## 3. What is a terminal state?

A **terminal state** is a state that represents the endpoint of the process.

For example:

Healthy  $\rightarrow$  Diseased  $\rightarrow$  Dead

If the model ends when the individual dies, then **Dead** is a terminal state.

Another example is a game:

Playing  $\rightarrow$  Won

Once the player wins, the game ends.

Therefore, **Won** is a terminal state.

## 4. What is an absorbing state?

An **absorbing state** is a state that, once entered, cannot be left.

Mathematically, a state  $i$  is absorbing if

$$p_{ii} = 1$$

That means:

$$P(X_{t+1} = i \mid X_t = i) = 1.$$

In simple words:

If the system is in state  $i$ , it will remain in state  $i$  forever.

Because the total probability leaving a state must equal 1, all other transition probabilities from that state must be zero:

$$p_{ij} = 0, \quad j \neq i$$

Therefore, an absorbing state has:

$$p_{ii} = 1$$

and

$$p_{ij} = 0 \quad (j \neq i).$$

## 5. Why can a terminal state be represented as an absorbing state?

Suppose we have:

$$S_1 \rightarrow S_2 \rightarrow S_3$$

and  $S_3$  is terminal.

Conceptually, the process stops once it reaches  $S_3$ .

However, a Markov chain is often represented as a process that continues indefinitely. To represent the terminal state mathematically, we therefore define:

$$S_3 \rightarrow S_3.$$

So instead of saying:

$$S_3 \rightarrow \text{END}$$

we represent the terminal state as:

$$S_3 \rightarrow S_3.$$

The corresponding probability is:

$$P(S_3 \rightarrow S_3) = 1.$$

Therefore:

$S_3$  is an absorbing state

# Terminal State as an Absorbing State

## 6. Example

Suppose a patient can be in three states:

- $S_1$ : Healthy
- $S_2$ : Diseased
- $S_3$ : Dead

Assume the transition matrix is

$$P = \begin{pmatrix} 0.7 & 0.2 & 0.1 \\ 0.1 & 0.6 & 0.3 \\ 0 & 0 & 1 \end{pmatrix}.$$

Let's interpret every element.

### First row

$$(0.7, 0.2, 0.1)$$

means that if the patient is healthy:

$$P(S_1 \rightarrow S_1) = 0.7$$

$$P(S_1 \rightarrow S_2) = 0.2$$

$$P(S_1 \rightarrow S_3) = 0.1.$$

So there is:

- 70% probability of remaining healthy
- 20% probability of becoming diseased
- 10% probability of moving directly to death

Notice:

$$0.7 + 0.2 + 0.1 = 1.$$

## 7. Second row

The second row is:

$$(0.1, 0.6, 0.3).$$

This means:

$$P(S_2 \rightarrow S_1) = 0.1$$

$$P(S_2 \rightarrow S_2) = 0.6$$

$$P(S_2 \rightarrow S_3) = 0.3.$$

Thus, if the patient is diseased:

- 10% probability of becoming healthy
- 60% probability of remaining diseased
- 30% probability of dying

Again:

$$0.1 + 0.6 + 0.3 = 1.$$

## 8. Third row: the absorbing state

The third row is:

$$(0, 0, 1).$$

This means:

$$P(S_3 \rightarrow S_1) = 0$$

$$P(S_3 \rightarrow S_2) = 0$$

and

$$P(S_3 \rightarrow S_3) = 1.$$

Therefore, once the patient reaches the **Dead** state, the model does not allow a transition back to Healthy or Diseased.

The process remains in:

$$S_3 \rightarrow S_3 \rightarrow S_3 \rightarrow \dots$$

This is why  $S_3$  is called **absorbing**.

### 13. Markov property

The concept is normally used within a **Markov chain**.

The Markov property states:

$$P(X_{t+1} = j \mid X_t = i, X_{t-1}, X_{t-2}, \dots) = P(X_{t+1} = j \mid X_t = i).$$

In words:

Given the current state, the next state does not depend on the complete previous history.

For an absorbing state  $k$ :

$$P(X_{t+1} = k \mid X_t = k) = 1.$$

It does not matter how the system arrived at  $k$ .

The general return can be written as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where:

- $G_t$  = return from time  $t$
- $R_{t+k+1}$  = future reward
- $\gamma$  = discount factor

For episodic tasks, we can alternatively write:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

## Important conventions

The book allows:

- $\gamma = 1$  when the sum remains defined, such as when episodes terminate.
- $T = \infty$  for continuing tasks.

But in the second formulation,  $T = \infty$  and  $\gamma = 1$  cannot both be used.

## Key takeaway

One notation can describe both episodic and continuing reinforcement-learning tasks.

## Markov Property

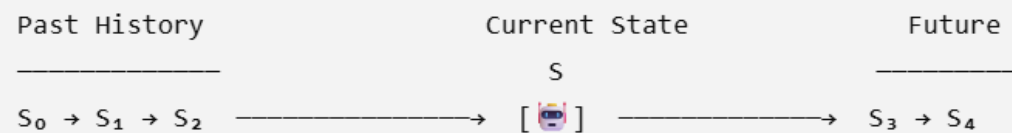
The Markov Property means:

The current state contains all the information needed to predict what happens next.

In other words, we do not need the entire history of the agent.

## Simple Example

Imagine a robot at position  $S$ :



If the current state  $S$  already tells us everything important, then knowing:

$$S_0, S_1, S_2$$

is unnecessary.

## Mathematical Definition

The Markov Property is:

$$P(S_{t+1}, R_{t+1} \mid S_t) = P(S_{t+1}, R_{t+1} \mid S_0, A_0, R_1, \dots, S_t, A_t)$$

## Meaning of the Variables

- $S_t$  = current state
- $A_t$  = current action
- $R_{t+1}$  = next reward
- $S_{t+1}$  = next state

The equation says:

The probability of the **next state and reward** depends only on the **current state and action**, not on the complete past history.

**Markov Property = “The present contains enough information about the past to predict the future.”**

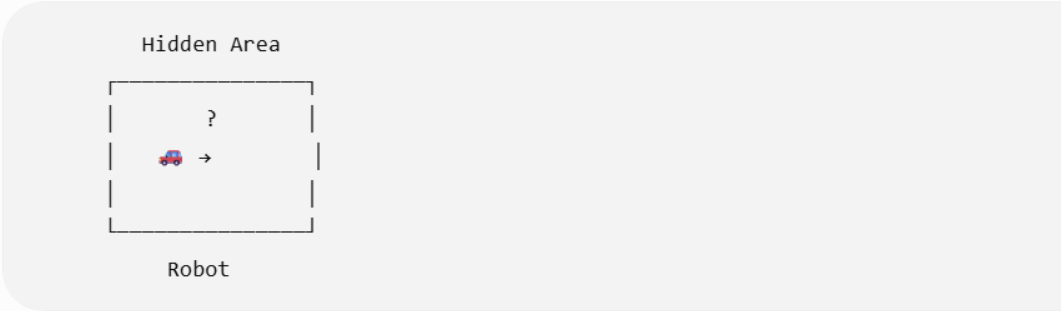
### Hidden Information

Hidden information means that the agent **cannot directly observe all the information needed to describe the true state of the environment.**

This can cause the **Markov Property to fail** if the agent's observation does not contain enough information about the current situation.

Example: Robot in a Maze 🤖

Imagine a robot can see only the room directly in front of it.

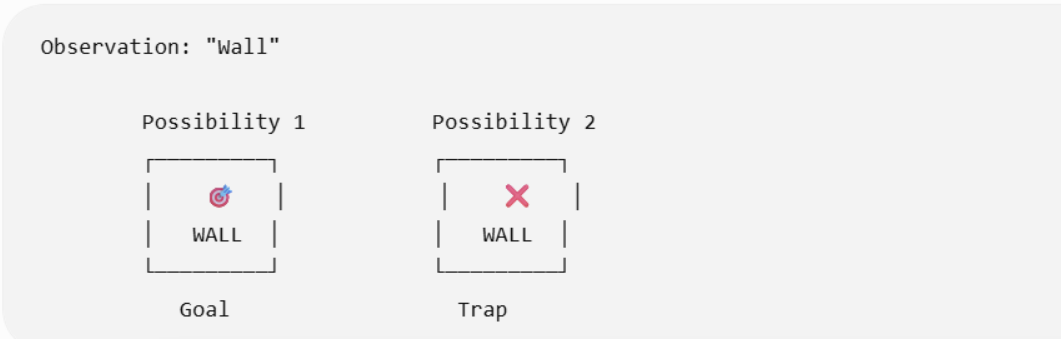


The robot sees:

$$O_t = \text{"wall in front"}$$

But it **does not know** what is behind the wall.

There could be:



The same observation can correspond to **different true states**.

Why Is This a Problem?

If the robot cannot distinguish these situations, then:

Observation  $\neq$  Complete State

Therefore, knowing only the current observation may **not be enough to predict the future**.

# Example: Pole-Balancing State

In **pole balancing**, the agent needs enough information about the cart and pole to predict what will happen next.

## 1. The State

A useful state can be represented as:

Markov Property

$s = (x, \dot{x}, \theta, \dot{\theta})$

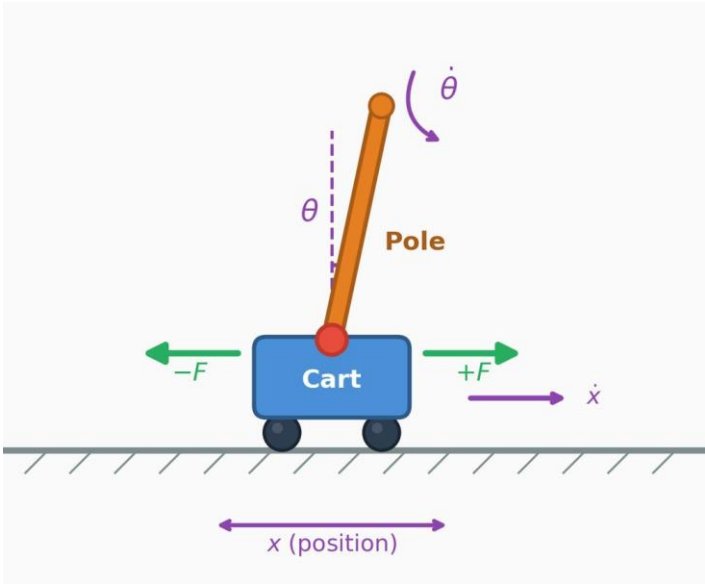
where:

| Variable       | Meaning               |
|----------------|-----------------------|
| $x$            | Cart position         |
| $\dot{x}$      | Cart velocity         |
| $\theta$       | Pole angle            |
| $\dot{\theta}$ | Pole angular velocity |

then the current state contains the important information needed to predict the next state:

$P(S_{t+1}, R_{t+1} | S_t, A_t)$

The previous history is not necessary.



Suppose at time  $t$ :

$$x_t = 2m$$

$$\dot{x}_t = 0.5 \text{ m/s}$$

$\theta_t = 10^\circ$

$$\dot{\theta}_t = 3^\circ/s$$

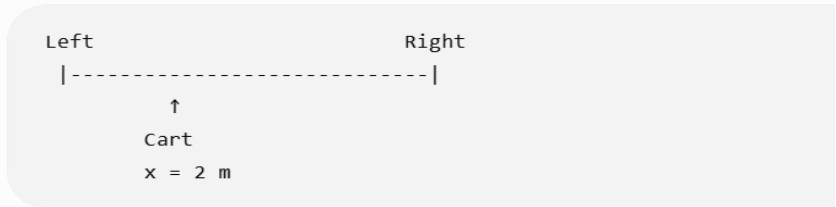
Therefore:

$$S_t = (2, 0.5, 10^\circ, 3^\circ/s)$$

This vector completely describes the current situation for our simplified model.

$x_t$ : Cart Position

This tells us where the cart is.



For example:

$$x_t = 2m$$

means the cart is 2 meters from the chosen reference point.

 $\dot{x}_t$ : Cart Velocity

This tells us how fast the cart is moving and in which direction.

For example:

$$\dot{x}_t = 0.5 \text{ m/s}$$

means the cart is moving to the right at  $0.5 \text{ m/s}$ .

If:

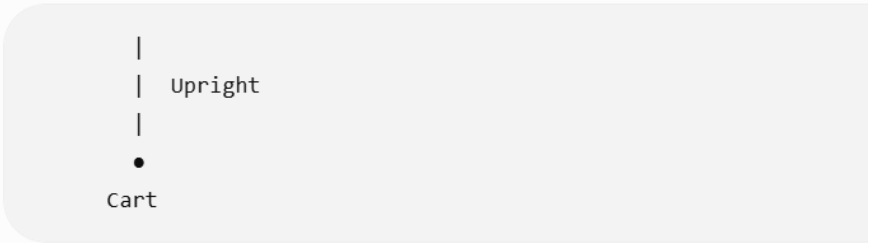
$$\dot{x}_t = -0.5 \text{ m/s}$$

the cart is moving in the opposite direction.

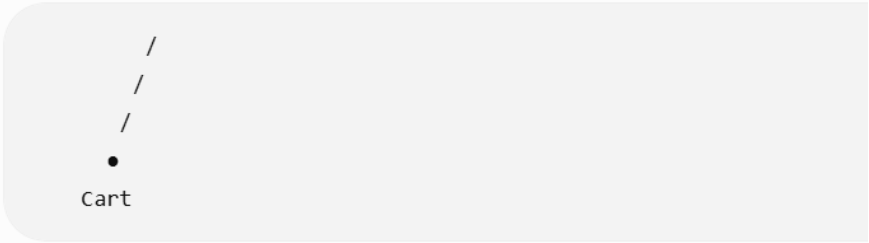
# Example: Pole-Balancing State

## $\theta_t$ : Pole Angle

This tells us how far the pole is tilted.



If the pole is tilted:



then:

$$\theta_t = 10^\circ$$

might mean the pole is tilted  $10^\circ$  from the vertical.

## $\dot{\theta}_t$ : Angular Velocity

This tells us how quickly the pole is rotating.

For example:

$$\dot{\theta}_t = 3^\circ/s$$

means the pole is currently rotating at  $3^\circ/s$ .

This variable is extremely important.

Consider two situations:

Situation A



$\theta = 10^\circ$   
 $\dot{\theta} = 0^\circ/\text{s}$

Situation B



$\theta = 10^\circ$   
 $\dot{\theta} = 5^\circ/\text{s}$

In both situations:

$$\theta = 10^\circ$$

But in Situation B, the pole is **moving quickly**.

Therefore, the agent may need to choose a different action.

So knowing only:

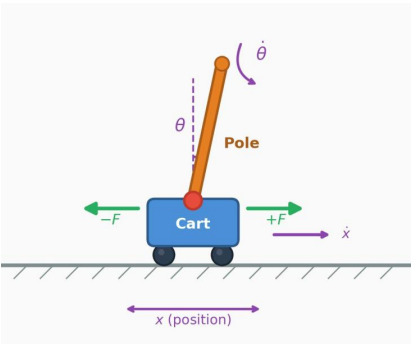
$$\theta$$

is not enough.

We also need:

$$\dot{\theta}$$

The same idea applies to the cart: position alone is not enough; its velocity also matters.



5. Connection to the Markov Property

The Markov Property says that the current state should contain enough information to predict the future.

Mathematically:

$P(S_{t+1}, R_{t+1} \mid S_t, A_t)$

The next state and reward depend on the **current state and action**, rather than requiring the entire history.

If:

$$S_t = (x_t, \dot{x}_t, \theta_t, \dot{\theta}_t)$$

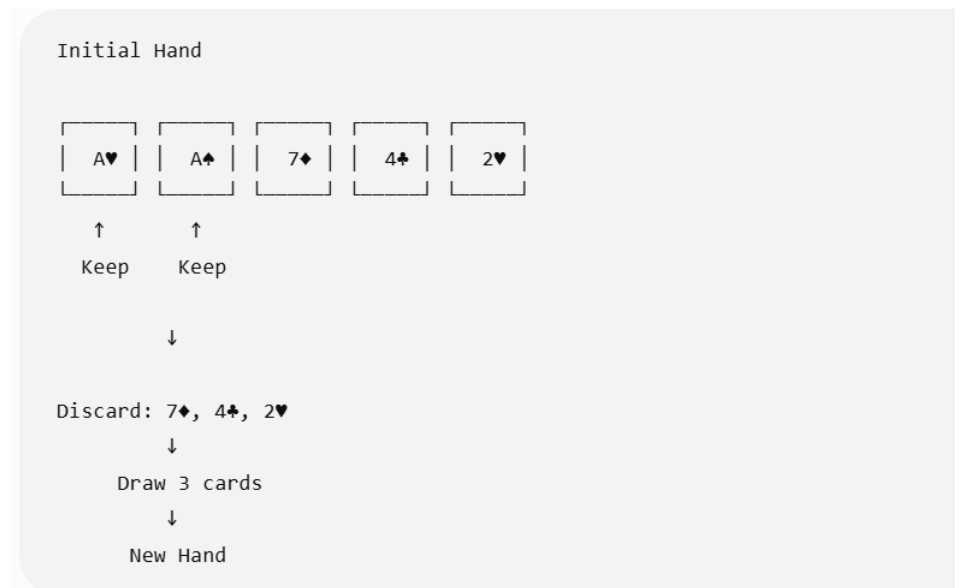
contains all the necessary information, then:

Current State + Action → Next State + Reward

**Draw Poker** is a classic example used in Reinforcement Learning to explain **states, actions, rewards, and the Markov property**.

## 1. The Basic Idea

In Draw Poker, the player receives a hand of cards and can choose which cards to **keep** and which cards to **discard and replace**.



## Exercise 2

The objective is to obtain a strong poker hand and receive a high reward.

A reinforcement-learning task that satisfies the **Markov property** is called a:

## Markov Decision Process (MDP)

If the state and action spaces are finite:

### Finite MDP

A finite MDP is defined by:

1. States
2. Actions
3. One-step dynamics

The central probability is:

$$p(s', r | s, a)$$

where:

- $s$  = current state
- $a$  = current action
- $s'$  = next state
- $r$  = reward

The complete dynamics are:

$$p(s', r | s, a) = P(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a)$$

This tells us:

Given the current state and action, what is the probability of each possible next state and reward?

### Why is this important?

These probabilities completely specify the dynamics of a finite MDP.

From them, we can calculate:

- Expected rewards
- Transition probabilities
- Expected reward for particular transitions

**Expected Reward** is the **average reward we expect to receive** when an action can lead to different outcomes.

In Reinforcement Learning, the result of an action is not always certain. Therefore, we calculate the expected reward using **probabilities**.

## 1. Basic Formula

The expected reward is:

$$\mathbb{E}[R] = \sum_i P_i R_i$$

Where:

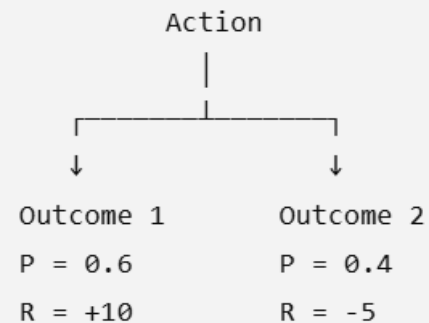
- $P_i$  = probability of outcome  $i$
- $R_i$  = reward for outcome  $i$
- $\mathbb{E}[R]$  = expected reward

In simple words:

**Expected Reward = Probability × Reward, summed over all possible outcomes.**

## 2. Simple Example

Suppose an agent chooses an action, and there are two possible outcomes:



Calculate the expected reward:

$$\begin{aligned}\mathbb{E}[R] &= (0.6)(10) + (0.4)(-5) \\ &= 6 - 2\end{aligned}$$

$$\boxed{\mathbb{E}[R] = 4}$$

So although the agent will **not necessarily receive exactly +4**, the long-run average reward is expected to be **+4 per trial**.

**State Transition Probability** describes the probability that the environment will move from the **current state** to a **particular next state** after the agent takes an action.

In simple words:

It tells us how likely each possible next state is.

The transition probability is:

$$p(s'|s, a) = P(S_{t+1} = s' | S_t = s, A_t = a)$$

It can be calculated from the complete dynamics:

$$p(s'|s, a) = \sum_r p(s', r | s, a)$$

## Meaning

It answers:

"If I take action  $a$  in state  $s$ , how likely am I to end up in state  $s'$ ?"

## 1. Basic Idea

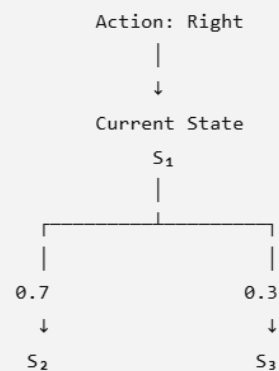
In Reinforcement Learning:

$$S_t + A_t \rightarrow S_{t+1}$$

But the next state is not always certain.

For example, a robot is in state  $S_1$  and chooses **Right**.

It might move to  $S_2$  with probability 0.7 or  $S_3$  with probability 0.3.



Therefore:

$$P(S_{t+1} = S_2 \mid S_t = S_1, A_t = \text{Right}) = 0.7$$

and:

$$P(S_{t+1} = S_3 \mid S_t = S_1, A_t = \text{Right}) = 0.3$$

Sometimes we want the expected reward given:

- Current state  $s$
- Action  $a$
- Next state  $s'$

This is:

$$r(s, a, s') = E[R_{t+1} | S_t = s, A_t = a, S_{t+1} = s']$$

and:

$$r(s, a, s') = \frac{\sum_r r p(s', r | s, a)}{p(s' | s, a)}$$

Three useful quantities

$$p(s', r | s, a)$$

Complete dynamics

$$p(s' | s, a)$$

Transition probability

$$r(s, a, s')$$

Expected reward for a transition

## States

$$S = \{high, low\}$$

The state represents the robot's battery level.

## Actions

When battery is high:

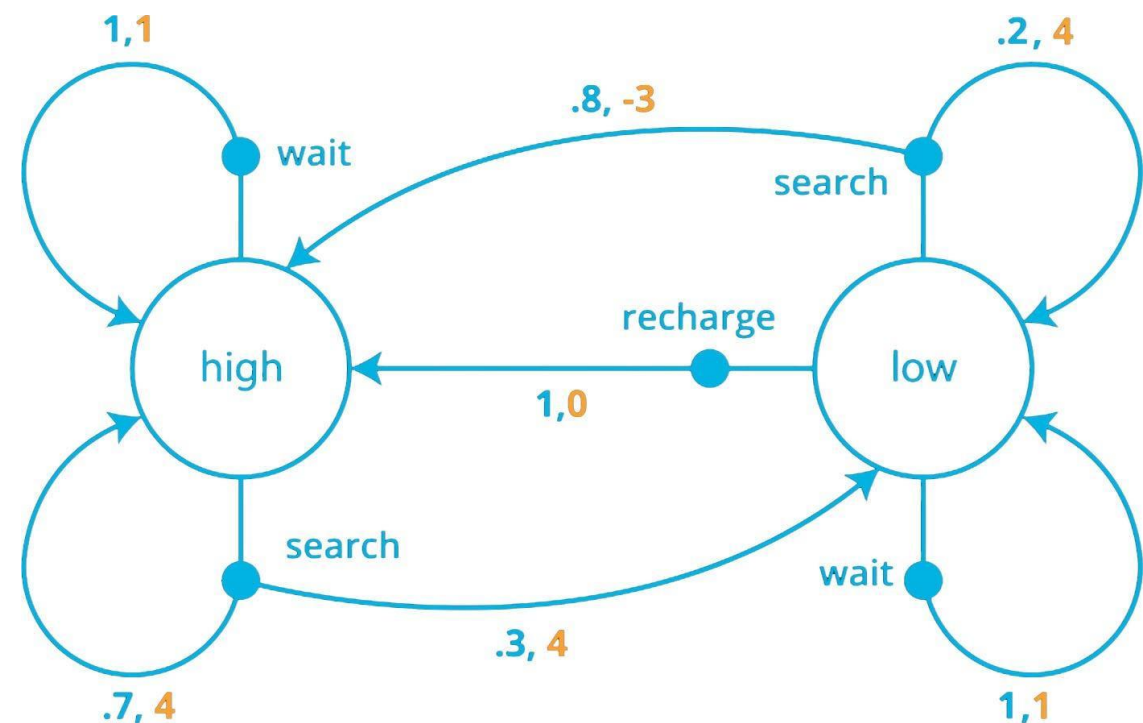
$$A(high) = \{search, wait\}$$

When battery is low:

$$A(low) = \{search, wait, recharge\}$$

## Why isn't recharge available at high?

Because recharging when already fully/high charged is unnecessary.



Waiting does not consume the battery.

For either energy level:

$$P(\text{same state} | s, \text{wait}) = 1$$

Expected reward:

$$r_{\text{wait}}$$

with:

$$r_{\text{search}} > r_{\text{wait}}$$

## Recharge

Only available in the low-energy state.

$$P(\text{high} | \text{low}, \text{recharge}) = 1$$

Reward:

$$R = 0$$

## Decision problem

The robot must balance:

**Search** → higher reward but battery risk

against

**Wait** → lower reward but safer

against

**Recharge** → restores energy but produces no reward

| Current state | Next state | Action   | Probability  | Expected reward |
|---------------|------------|----------|--------------|-----------------|
| high          | high       | search   | $\alpha$     | $r_{search}$    |
| high          | low        | search   | $1 - \alpha$ | $r_{search}$    |
| low           | high       | search   | $1 - \beta$  | $-3$            |
| low           | low        | search   | $\beta$      | $r_{search}$    |
| high          | high       | wait     | 1            | $r_{wait}$      |
| low           | low        | wait     | 1            | $r_{wait}$      |
| low           | high       | recharge | 1            | 0               |

**Important observation**

For every state-action pair:

$$\sum_{s'} p(s'|s, a) = 1$$

The transition probabilities leaving an action node must add up to 1.

## How to read the graph

Large open circles

→ States

Small solid circles

→ State-action pairs

Arrows

→ Possible transitions

Each arrow contains:

Probability, Expected reward

## Example

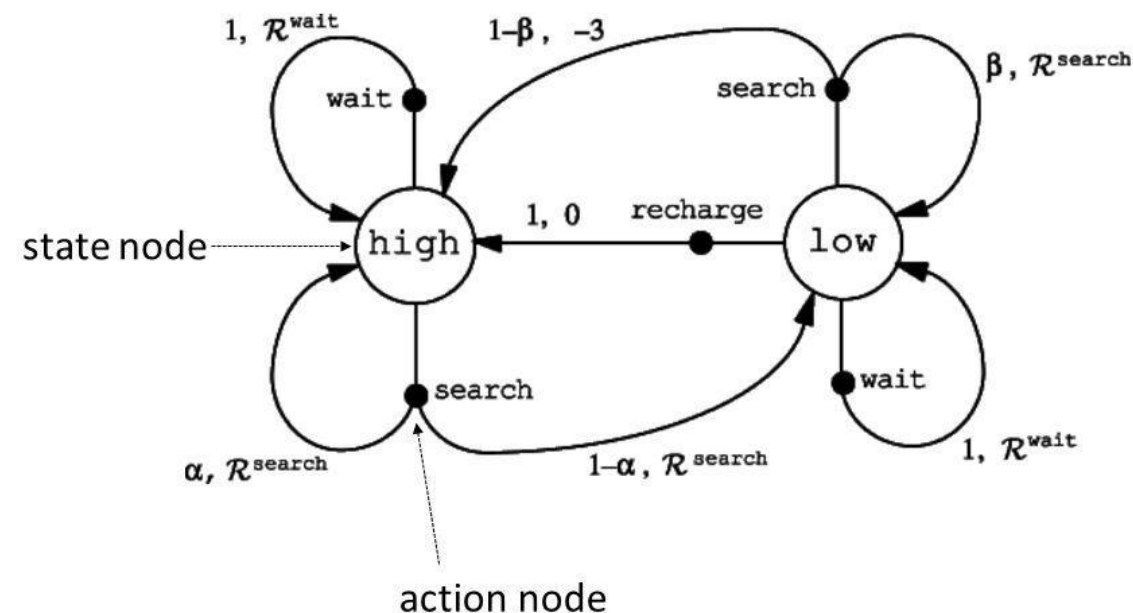
From **high** + **search**:

- Remain high with probability  $\alpha$
- Become low with probability  $1 - \alpha$

## Key idea

The transition graph provides a visual summary of the MDP dynamics.

## Example MDP



Almost all RL algorithms estimate **value functions**.

A value function tells us:

How good is a state or action in terms of expected future rewards?

The important word is:

### Future

A state is valuable not just because of its immediate reward, but because of the rewards that can be obtained afterward.

### Value depends on the policy

Future actions depend on the policy  $\pi$ .

Therefore:

$$v_{\pi}(s)$$

and

$$q_{\pi}(s, a)$$

are values **under policy**  $\pi$ .

The state-value function is:

$$v_{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

Expanded:

$$v_{\pi}(s) = E_{\pi} \left[ \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

### Simple meaning

$v_{\pi}(s)$  tells us the expected return when we start in state  $s$  and then follow policy  $\pi$ .

### Terminal state

$$v_{\pi}(\text{terminal}) = 0$$

because no further rewards are received.

The action-value function is:

$$q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

Simple meaning

$q_{\pi}(s, a)$  tells us the expected return when we start in state  $s$ , take action  $a$ , and then follow policy  $\pi$ .

Difference

|            | State Value             | Action Value                           |
|------------|-------------------------|----------------------------------------|
| Function   | $v_{\pi}(s)$            | $q_{\pi}(s, a)$                        |
| Question   | How good is this state? | How good is this action in this state? |
| Depends on | Policy                  | Policy + selected action               |

Value functions can be estimated by observing actual experience.

## State-value estimation

For each state:

1. Visit state  $s$ .
2. Observe the return that follows.
3. Record the return.
4. Average the observed returns.

With many visits:

$$\text{Average Returns} \rightarrow v_{\pi}(s)$$

## Action-value estimation

Do the same for each state-action pair:

$$\text{Average Returns} \rightarrow q_{\pi}(s, a)$$

These approaches are called:

## Monte Carlo methods

because they estimate values using many sampled experiences.

If there are only a small number of states:

- We can maintain a separate value for each state.

But real problems may have:

- Thousands
- Millions
- Or extremely large numbers of states.

It becomes impractical to store one value per state.

### Solution

Use a **parameterized function approximator**:

$$v_{\pi}(s; \mathbf{w})$$

or

$$q_{\pi}(s, a; \mathbf{w})$$

and adjust its parameters based on experience.

### Key idea

Instead of memorizing every state separately, learn a function that can estimate values for many states.

## Bellman Equation for $v_\pi$

Bellman equation = the mathematical foundation of the backup process.

### Bellman Equation

The **Bellman equation** tells us how to calculate the value of a state from its **immediate reward + future value**.

### Main Formula

$$V_\pi(s) = \sum_a \pi(a|s) [R + \gamma V_\pi(s')]$$

### Meaning of the Variables

- $V_\pi(s)$  = value of the **current state**
- $s$  = current state
- $a$  = action
- $\pi(a|s)$  = probability of choosing action  $a$
- $R$  = immediate reward
- $\gamma$  = discount factor
- $s'$  = next state
- $V_\pi(s')$  = value of the next state

Example— Backup Diagrams

Example

Imagine a robot is at **State S**:

- Given
- Probability of Action A = 0.5  $\pi(a|s)$
  - Probability of Action B = 0.5  $\pi(b|s)$
  - Discount factor:  $\gamma = 0.9$
  - $V(S_1) = 4$
  - $V(S_2) = 2$

Step 1: Calculate Action A

$$R + \gamma V(S_1)$$
$$= 0 + 0.9(4) = 3.6$$

So, Action A  $\rightarrow$  3.6

Step 2: Calculate Action B

$$R + \gamma V(S_2)$$
$$= -1 + 0.9(2) = 0.8$$

So, Action B  $\rightarrow$  0.8

Bellman Equation for  $v_\pi$

Step 3: Backup to State S

The general formula is:

$$V_\pi(s) = \sum_a \pi(a|s) [R + \gamma V_\pi(s')]$$

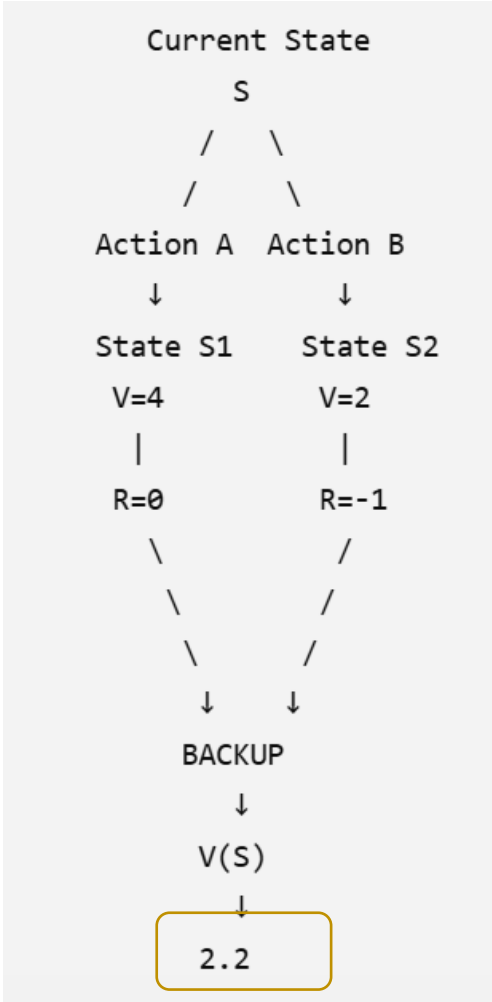
Therefore:

$$R_A = 0, \quad R_B = -1$$

$$V(S) = 0.5[0 + 0.9(4)] + 0.5[-1 + 0.9(2)]$$

$$V(S) = 1.8 + 0.4$$

$$V(S) = 2.2$$



## Gridworld Environment

**Gridworld** is a simple environment used in Reinforcement Learning to explain how an agent learns to choose actions.

### 1. Environment

The environment is a **grid of cells**:

```
+---+---+---+---+
|   |   |   | G |
+---+---+---+---+
|   | X |   |   |
+---+---+---+---+
| S |   |   |   |
+---+---+---+---+
```

- S = Starting state
- G = Goal/terminal state
- X = obstacle
- Empty cells = possible states

### 2. State

The **state** is the current position of the agent.

For example:

$$s = (2, 1)$$

means the agent is at row 2, column 1.

### 3. Actions

At each state, the agent can usually move:

$$A = \{\text{Up, Down, Left, Right}\}$$

If an action would move the agent outside the grid or into an obstacle, that move may not be allowed.

## 4. Reward

The environment gives a reward after each action.

For example:

- Normal movement  $\rightarrow -1$
- Reaching the goal  $\rightarrow 0$  or a positive reward, depending on the problem.

If the agent reaches the goal in 4 steps:

$$G = -1 - 1 - 1 - 1 = -4$$

Therefore, the agent tries to reach the goal using **as few steps as possible**.

## 5. Value Function

The value of a state tells us how good it is to be in that state.

For example:

$$v(s) = -3$$

means that, under the current policy, the agent expects about **3 more steps** to reach the goal.

## Example

The agent starts at:

$$S = (4, 1)$$

The goal is:

$$G = (1, 4)$$

One possible path is:

$$(4, 1) \rightarrow (3, 1) \rightarrow (2, 1) \rightarrow (1, 1) \rightarrow (1, 2) \rightarrow (1, 3) \rightarrow (1, 4)$$

So, the agent needs **6 moves**.

If the reward for each move is:

$$R = -1$$

then the total return is:

$$G = -1 - 1 - 1 - 1 - 1 - 1$$

$$G = -6$$

|       | Column |   |   |   |
|-------|--------|---|---|---|
|       | 1      | 2 | 3 | 4 |
| Row 1 | 0      | 1 | 2 | G |
| Row 2 | 1      | X | 3 | 4 |
| Row 3 | 2      | 3 | 4 | 5 |
| Row 4 | 5      | 5 | 6 | 7 |

The numbers can represent the **number of moves needed to reach the goal**:

- 0 → Goal
- 1 → One move from the goal
- 2 → Two moves from the goal
- 3 → Three moves from the goal
- etc.

X represents an obstacle, so the agent cannot move into that cell.

RL Components

State = Agent's position

Actions = {Up, Down, Left, Right}

Reward = -1 for each move

Goal = Reach G

**Main idea:** The agent learns to find a path to the goal that uses fewer moves, because every move gives a penalty of -1.

|       | Column |   |   |   |
|-------|--------|---|---|---|
|       | 1      | 2 | 3 | 4 |
| Row 1 | 0      | 1 | 2 | G |
| Row 2 | 1      | X | 3 | 4 |
| Row 3 | 2      | 3 | 4 | 5 |
| Row 4 | 5      | 5 | 6 | 7 |

## Goal

Get the ball into the hole.

## Reward

Each stroke:

$$R = -1$$

Therefore:

Fewer strokes  $\rightarrow$  higher return.

## State

The location of the ball.

## Actions

In golf, an action includes decisions such as:

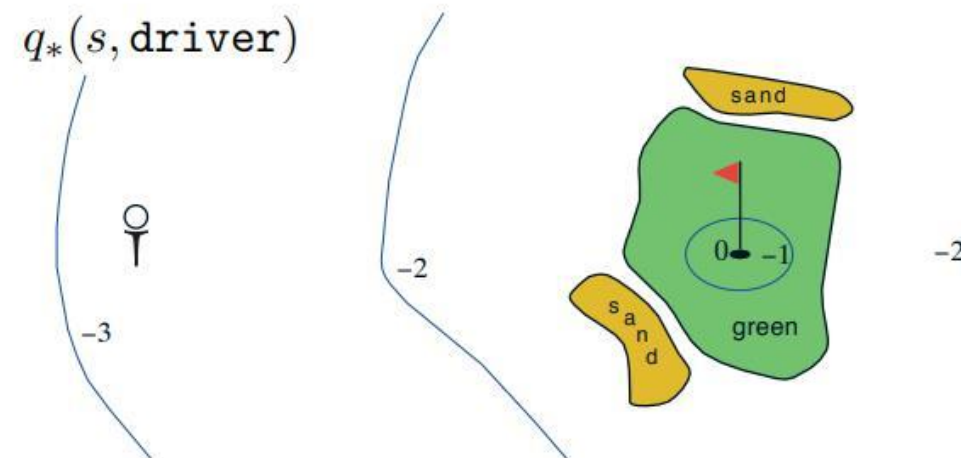
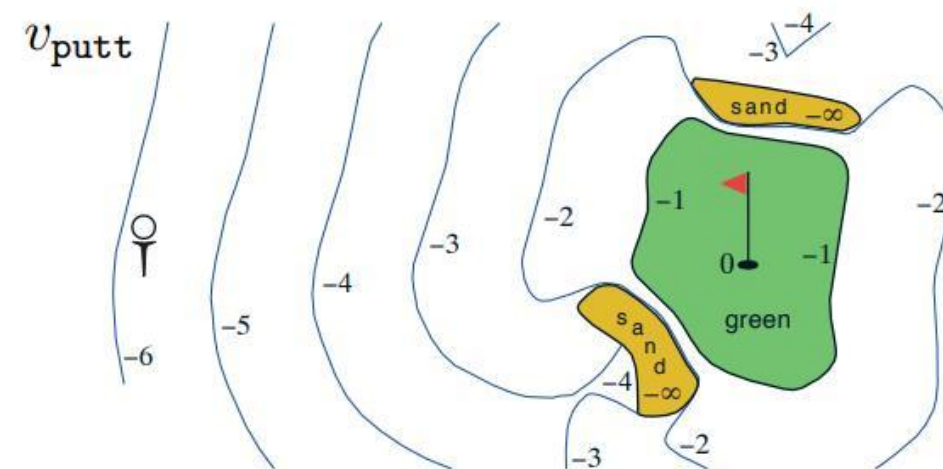
- how to aim,
- how strongly to swing,
- and which club to use.

However, in this example, the authors simplify the problem.

They assume that **aiming and swinging are already determined**, and only consider the choice of club.

There are two possible actions:

$$A = \{\text{putter, driver}\}$$



Putter policy

Suppose the policy always uses a putter.

The value function is:

$$v_{\text{putt}}(s)$$

Terminal state

Ball in hole:

$$v_{\text{putt}}(s) = 0$$

On the green

If the ball can reach the hole in one putt:

$$v_{\text{putt}}(s) = -1$$

Locations requiring two strokes

$$v_{\text{putt}}(s) = -2$$

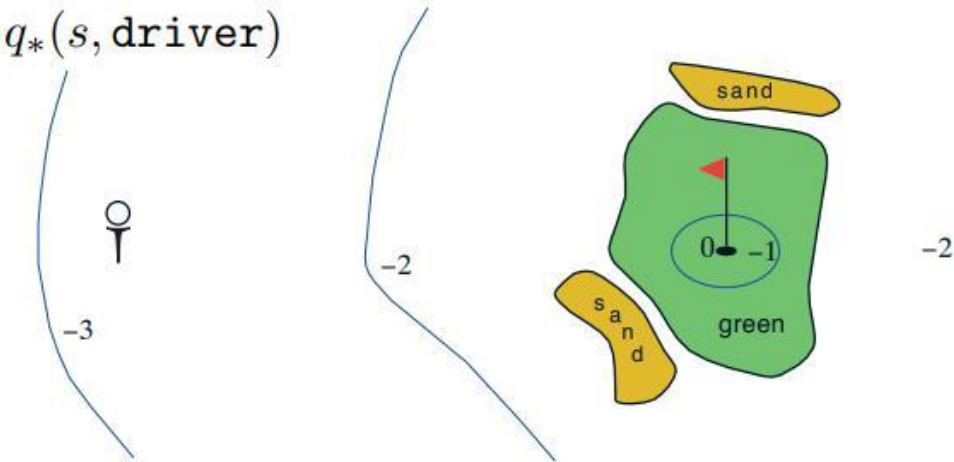
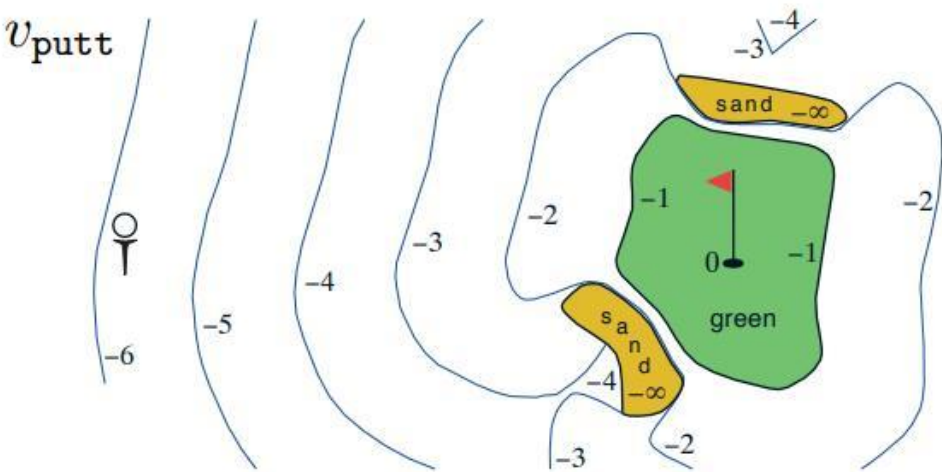
Three strokes:

$$v_{\text{putt}}(s) = -3$$

and so on.

Interpretation

The contour lines represent the expected number of strokes needed to reach the hole.



The lower part of Figure 3.6 shows:

$$q^*(s, \text{driver})$$

This asks:

“What is the expected return if I choose the driver from state  $s$  and then act optimally afterward?”

Difference

$$v_{\text{putt}}(s)$$

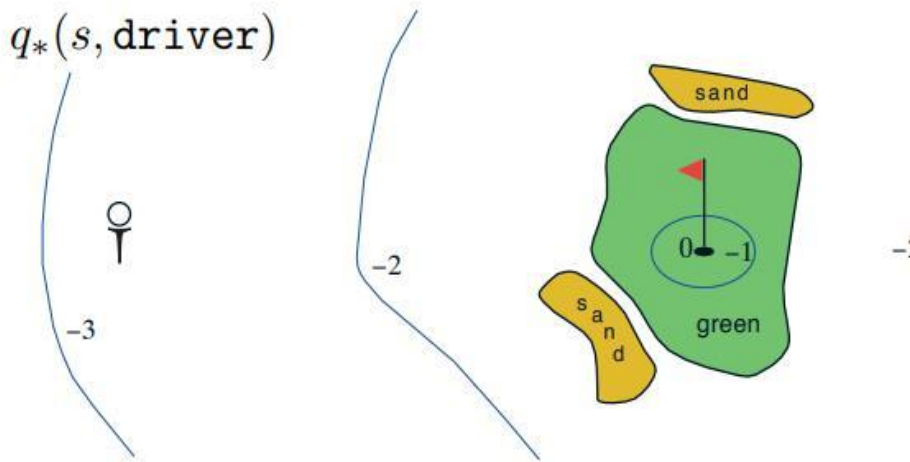
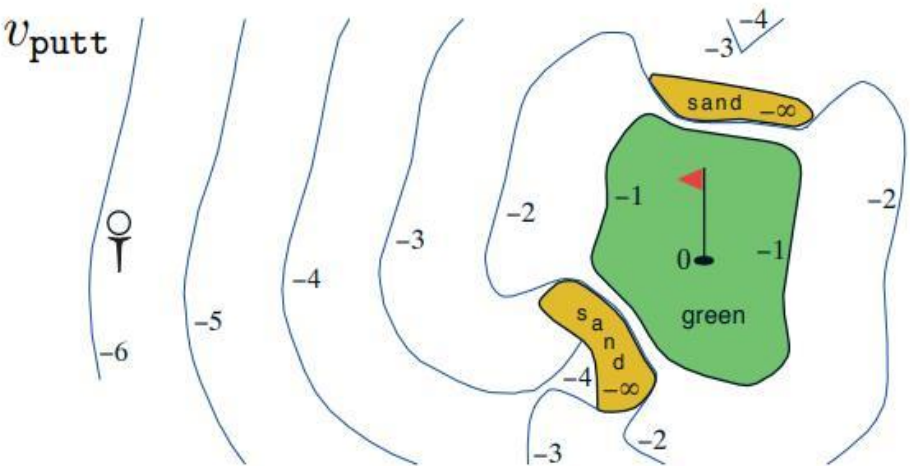
→ value when following the **putter** policy

$$q^*(s, \text{driver})$$

→ value of choosing **driver** and then behaving optimally.

Main lesson

Different actions can have very different long-term consequences even when their immediate reward is similar.



## 1. Define the RL problem

We model playing golf as a Reinforcement Learning problem.

- **State  $s$ :** the current location of the ball.
- **Actions:** choose a **putter** or a **driver**.
- **Reward:**  $-1$  for every stroke.
- **Terminal state:** the ball reaches the hole.
- **Terminal value:**

$$v(\text{hole}) = 0$$

## 2. Value of a state

Because every stroke gives  $-1$ , the value of a state is the negative number of strokes needed to reach the hole.

For example:

$$1 \text{ stroke} \Rightarrow v(s) = -1$$

$$2 \text{ strokes} \Rightarrow v(s) = -2$$

$$3 \text{ strokes} \Rightarrow v(s) = -3$$

So, fewer strokes means a higher value.

## 3. If we always use the putter

Suppose the policy is:

$$\pi(s) = \text{putter}$$

From anywhere on the green, we can reach the hole in one putt.

Therefore:

$$v_{\text{putt}}(s) = -1$$

for states on the green.

If a state requires two putts:

$$v_{\text{putt}}(s) = -2$$

If it requires three putts:

$$v_{\text{putt}}(s) = -3$$

and so on.

## 4. What happens in the sand?

Under the assumptions of the example, the putter cannot get the ball out of a sand trap.

Therefore, the hole can never be reached from there if we always use the putter:

$$v_{\text{putt}}(s) = -\infty$$

## 5. From the tee

The example assumes that using only the putter requires **6 strokes** to reach the hole.

Therefore:

$$v_{\text{putt}}(\text{tee}) = -6$$

## 6. Why do we consider the driver?

The driver can move the ball much farther and can help us escape difficult locations.

Now we evaluate an action using:

$$q^*(s, a)$$

For example:

$$q^*(s, \text{driver})$$

means:

Take the driver now, then make optimal decisions afterward.

## 7. Optimal decision

At each state, we compare the available actions:

$$q^*(s, \text{putter})$$

and

$$q^*(s, \text{driver})$$

Then choose the action with the **larger value**.

The optimal state value is:

$$v^*(s) = \max_a q^*(s, a)$$

For example, if:

$$q^*(s, \text{putter}) = -4$$

and

$$q^*(s, \text{driver}) = -2$$

then:

$$v^*(s) = \max(-4, -2) = -2$$

So the optimal choice in this state is the action that leads to a total of **2 strokes**.

## Final answer

The golf example shows that:

$$\text{State} = \text{ball location}$$

$$\text{Action} = \text{putter or driver}$$

$$\text{Reward} = -1 \text{ per stroke}$$

$$v_{\text{putt}}(s) = \text{negative number of strokes to the hole}$$

and

$$v^*(s) = \max_a q^*(s, a)$$

**Main idea:** The agent chooses actions based not only on the immediate reward, but also on how the action affects the future.

- An RL agent aims to maximize its expected long-term reward.
- An **optimal policy** is a policy that performs at least as well as every other policy.
- Optimal value functions tell us:
  - How valuable each state is.
  - How valuable each state–action pair is.
- The main concepts:
  - Optimal policy  $\pi^*$
  - Optimal state-value function  $v^*$
  - Optimal action-value function  $q^*$
  - Bellman optimality equations

**Key idea:**

Optimal value functions tell us how good the best possible future can be.

## What is a Policy?

A policy  $\pi(a|s)$  specifies the probability of choosing action  $a$  when the agent is in state  $s$ .

## State-Value Function

The value of a state under policy  $\pi$  is:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

where  $G_t$  is the return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

## Interpretation

$v_{\pi}(s)$  answers:

"If I am in state  $s$  and continue following policy  $\pi$ , how much reward can I expect in the long run?"

Two policies can be compared using their value functions.

Policy  $\pi_1$  is better than or equal to  $\pi_2$  if:

$$v_{\pi_1}(s) \geq v_{\pi_2}(s)$$

for every state  $s$ .

Example

Suppose:

| State | $v_{\pi_1}$ | $v_{\pi_2}$ |
|-------|-------------|-------------|
| $s_1$ | 10          | 8           |
| $s_2$ | 7           | 5           |
| $s_3$ | 4           | 4           |

Then:

$$\pi_1 \geq \pi_2$$

because  $\pi_1$  is at least as good in every state.

Important

There can be more than one optimal policy.

An **optimal policy**  $\pi^*$  is a policy that is at least as good as every other policy.

$$v_{\pi^*}(s) \geq v_{\pi}(s)$$

for every policy  $\pi$  and every state  $s$ .

There may be multiple optimal policies.

## Example

Suppose in state  $s$ :

- Action A gives value 10
- Action B gives value 10
- Action C gives value 6

Both policies that choose A or B can be optimal.

Therefore:

Optimal does not necessarily mean unique.

The optimal state-value function is:

$$v^*(s) = \max_{\pi} v_{\pi}(s)$$

It represents the maximum expected return that can be obtained from state  $s$ .

In simple words

$$v^*(s)$$

means:

"What is the maximum long-term reward I can get if I start from state  $s$  and make the best decisions?"

**Example**

If:

$$v_{\pi_1}(s) = 5$$

$$v_{\pi_2}(s) = 8$$

$$v_{\pi_3}(s) = 6$$

then:

$$v^*(s) = 8$$

The **optimal action-value function** is:

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

It represents the expected return when:

1. We are in state  $s$ .
2. We first take action  $a$ .
3. After that, we follow an optimal policy.

### Important distinction

$v^*(s)$ :

"What is the best value I can get from state  $s$ ?"

$q^*(s, a)$ :

"What happens if I choose this specific action first and then behave optimally?"

The optimal state value can be obtained by choosing the best action:

$$v^*(s) = \max_a q^*(s, a)$$

## Example

Suppose:

$$q^*(s, A) = 10$$

$$q^*(s, B) = 7$$

$$q^*(s, C) = 4$$

Then:

$$v^*(s) = \max(10, 7, 4) = 10$$

Therefore, action A is an optimal action.

## Key idea

$q^*$  tells us the value of each action.

$v^*$  tells us the value of the best action.

The Bellman optimality equation is:

$$v^*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v^*(S_{t+1}) \mid S_t = s, A_t = a]$$

## Meaning

The optimal value of a state is:

Immediate Reward + Discounted Future Optimal Value

and we choose the action that gives the maximum expected value.

## In simple words

"Try every possible action, calculate its expected immediate reward plus future value, and choose the best one."

The equation

$$v^*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v^*(S_{t+1})]$$

has three important components.

## 1. Immediate Reward

$$R_{t+1}$$

Reward received after taking the action.

## 2. Future Value

$$\gamma v^*(S_{t+1})$$

The value of the next state, discounted by  $\gamma$ .

## 3. Maximum

$$\max_a$$

Choose the action with the highest expected total value.

Suppose an agent is in state  $s$  and has two actions.

## Action A

- Immediate reward = 5
- Future optimal value = 10
- $\gamma = 0.9$

$$Q_A = 5 + 0.9(10) = 14$$

## Action B

- Immediate reward = 8
- Future optimal value = 5

$$Q_B = 8 + 0.9(5) = 12.5$$

Therefore:

$$v^*(s) = \max(14, 12.5) = 14$$

## Optimal Action

A

The agent chooses A even though B gives a larger immediate reward.

Why?

Because A provides a better long-term return.

For the optimal action-value function:

$$q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a') \mid S_t = s, A_t = a]$$

## Interpretation

First:

Take action  $a$ .

Then:

Observe the next state.

Finally:

Choose the best possible action  $a'$  from the next state.

## Key difference

For  $q^*$ , the first action is **fixed**.

The future actions are chosen optimally.

$$v^*(s)$$

Value of a state

Chooses the best action

$$v^*(s) = \max_a q^*(s, a)$$

Requires comparing actions

$$q^*(s, a)$$

Value of a state-action pair

Evaluates a specific first action

Includes action  $a$  explicitly

Directly tells us which action is valuable

## Example

If:

$$q^*(s, A) = 15$$

$$q^*(s, B) = 10$$

then:

$$v^*(s) = 15$$

and A is an optimal action.

Imagine a golf player choosing between:

- **Driver:** hits the ball far but is less accurate.
- **Putter:** is accurate but only useful when close to the hole.

$q^*(s, \text{driver})$

This asks:

"What is the expected number of strokes if I use the driver now and then make optimal decisions afterward?"

## Important idea

The driver may not be the best action in every state.

For example:

- Very close to the hole → Putter may be optimal.
- Far from the hole → Driver may be optimal.

The value function considers the **future consequences** of the current action.

Suppose we know  $q^*(s, a)$ .

To choose the optimal action, we simply calculate:

$$a^* = \arg \max_a q^*(s, a)$$

## No model is required for action selection

The agent does **not** need to know:

- What the next states will be.
- Their transition probabilities.
- Their future values.

The action-value function already contains this information.

## Key idea

$q^*$  effectively "caches" the results of future searches.

A policy is **greedy with respect to a value function** if it always chooses the action with the highest estimated value.

For  $q^*$ :

$$\pi^*(s) = \arg \max_a q^*(s, a)$$

Example

| Action | $q^*(s, a)$ |
|--------|-------------|
| Left   | 5           |
| Right  | 12          |
| Up     | 8           |

The greedy action is:

Right

because:

$$12 > 8 > 5$$

Normally, a greedy decision only considers immediate consequences.

That can be dangerous.

### Example

Imagine:

- Action A  $\rightarrow$  immediate reward = 10, but future rewards are poor.
- Action B  $\rightarrow$  immediate reward = 2, but leads to large future rewards.

A short-sighted greedy strategy may choose A.

**But  $v^*$  is different.**

$v^*$  already includes the value of **all future optimal behavior**.

Therefore, when we evaluate the one-step consequences using  $v^*$ , choosing the best action is sufficient.

**A one-step greedy decision using  $v^*$  produces an optimal action.**

Because exact solutions are often impractical, RL algorithms usually seek **approximate solutions**.

Instead of calculating the exact:

$$v^*(s)$$

we estimate:

$$\hat{v}(s) \approx v^*(s)$$

Similarly:

$$\hat{q}(s, a) \approx q^*(s, a)$$

### Examples of approaches

- Dynamic Programming
- Monte Carlo methods
- Temporal-Difference learning
- Q-learning
- Function approximation
- Deep Reinforcement Learning

When the number of states is small, we can store one value for every state.

Example:

| State | Estimated Value |
|-------|-----------------|
| $s_1$ | 10.2            |
| $s_2$ | 8.7             |
| $s_3$ | 5.4             |
| $s_4$ | 2.1             |

This is called the **tabular case**.

## Advantages

- Simple
- Easy to understand
- Direct representation

## Disadvantage

It becomes impossible when the state space is extremely large.

For large state spaces, we cannot store a separate value for every state.

Instead, we use a parameterized function:

$$\hat{v}(s; \theta)$$

where:

- $s$  = state
- $\theta$  = parameters
- $\hat{v}$  = estimated value

## Example

A neural network can approximate:

$$\hat{q}(s, a; \theta) \approx q^*(s, a)$$

This is the basic idea behind many **Deep Reinforcement Learning** algorithms.

An agent does not necessarily need to behave perfectly in every possible state.

Some states may be:

- Very common → important to learn accurately.
- Very rare → less important for overall performance.

### Example

A game-playing agent may encounter certain positions thousands of times, while other theoretically possible positions may almost never occur.

RL can focus learning on states that are frequently encountered.

### Key idea

Online learning naturally allows the agent to spend more effort on states that matter most in its experience.

## 1. Optimal Policy

$$\pi^*$$

achieves the highest expected return.

## 2. Optimal State Value

$$v^*(s) = \max_{\pi} v_{\pi}(s)$$

tells us the best possible value from state  $s$ .

## 3. Optimal Action Value

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

tells us how good a specific first action is.

## 4. Bellman Optimality

$$v^*(s) = \max_a \mathbb{E}[R + \gamma v^*(S')]$$

## 5. Greedy Selection

$$a^* = \arg \max_a q^*(s, a)$$

## 6. Practical RL

Exact optimal solutions are often impossible, so RL usually learns **approximations**.

Optimal value functions allow an RL agent to convert a long-term decision problem into a sequence of local decisions.

- $v^*(s)$  → How good is this state?
- $q^*(s, a)$  → How good is this action in this state?
- Bellman optimality → Best immediate reward + best future value.
- Greedy with  $v^*$  → Optimal decision.
- Greedy with  $q^*$  → Direct optimal action selection.
- Exact solutions → Often computationally expensive.
- Approximation → Essential for large real-world problems.

$$\text{Optimal Action} = \arg \max_a q^*(s, a)$$

This is the central idea behind the section.

$$\pi(a|s) = P(A_t = a|S_t = s)$$

$$p(s', r|s, a) = P(S_{t+1} = s', R_{t+1} = r|S_t = s, A_t = a)$$

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$v^{\pi}(s) = \mathbb{E}_{\pi}[G_t|S_t = s]$$

$$q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t|S_t = s, A_t = a]$$

$$v^{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a)[r + \gamma v^{\pi}(s')]$$

$$v^*(s) = \max_{\pi} v^{\pi}(s)$$

$$v^*(s) = \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma v^*(s')]$$

# Example - Delivery Robot

Consider a delivery robot operating inside a small building. The robot must move from a warehouse to a destination where it has to deliver a package.

This problem is modeled as a **finite Markov Decision Process (MDP)**.

### 1. Environment

The robot can be in one of three states:

$$S = \{A, B, C\}$$

where:

- **State A:** The robot is near the warehouse and has just started its delivery.
- **State B:** The robot is in a hallway between the warehouse and the destination.
- **State C:** The robot has reached the destination.

The state diagram can be understood as:

$$A \rightarrow B \rightarrow C$$

However, because some actions are unsuccessful, the robot may remain in the same state.

### 1. Environment

The robot can be in one of three states:

$$S = \{A, B, C\}$$

where:

- **State A:** The robot is near the warehouse and has just started its delivery.
- **State B:** The robot is in a hallway between the warehouse and the destination.
- **State C:** The robot has reached the destination.

The state diagram can be understood as:

$$A \rightarrow B \rightarrow C$$

However, because some actions are unsuccessful, the robot may remain in the same state.

### 2. Available Actions

The available actions depend on the current state.

In state A:

$$A(A) = \{move, wait\}$$

The robot can either:

- **move:** Move toward the hallway.
- **wait:** Stay near the warehouse.

In state B:

$$A(B) = \{move, wait\}$$

The robot can either:

- **move:** Attempt to reach the destination.
- **wait:** Stay in the hallway.

In state C:

$$A(C) = \{wait\}$$

The robot has already reached the destination, so its only possible action is to wait.

### Questions

#### Question 1 — MDP Components

Identify the following components of the MDP:

1. The agent
2. The environment
3. The state space
4. The available actions
5. The possible rewards
6. The discount factor

## 3. Transition Dynamics and Rewards

The environment behaves according to the following rules.

### From State A

If the robot chooses **move**:

$$P(B, +2|A, move) = 1$$

This means:

- The robot definitely moves from  $A$  to  $B$ .
- It receives a reward of  $+2$ .

If the robot chooses **wait**:

$$P(A, 0|A, wait) = 1$$

So the robot:

- remains in  $A$ ,
- receives reward 0.

### From State B

If the robot chooses **move**, there are two possible outcomes.

#### Successful movement

With probability 0.8:

$$P(C, +10|B, move) = 0.8$$

The robot reaches the destination and receives:

$$R = +10$$

#### Failed movement

With probability 0.2:

$$P(B, -2|B, move) = 0.2$$

The robot fails to reach the destination, remains in  $B$ , and receives:

$$R = -2$$

Therefore, moving from  $B$  is uncertain.

### If the robot waits in B

$$P(B, 0|B, wait) = 1$$

The robot remains in  $B$  and receives no reward.

---

### From State C

The robot has completed its delivery.

$$P(C, 0|C, wait) = 1$$

It remains in  $C$  and receives reward 0.

### 4. Discount Factor

Assume the discount factor is:

$$\gamma = 0.9$$

The discount factor determines how much the agent values future rewards.

The return is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Because:

$$\gamma = 0.9 < 1$$

future rewards are worth less than immediate rewards.

For example, a reward of 10 received one step later has discounted value:

$$0.9(10) = 9$$

while a reward of 10 received two steps later has value:

$$0.9^2(10) = 8.1$$

# 5. The Agent–Environment Interaction

At every time step, the interaction follows:

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1}$$

For example:

$$A \xrightarrow{\text{move}} B$$

and the environment produces:

## Example - Delivery Robot

So we can write:

$$S_t = A$$

$$A_t = \text{move}$$

$$R_{t+1} = +2$$

$$S_{t+1} = B$$

Then the robot observes the new state  $B$  and chooses its next action.

# 6. The Policy

Suppose the robot follows the following policy:

$$\pi(\text{move}|A) = 1$$

$$\pi(\text{move}|B) = 0.5$$

$$\pi(\text{wait}|B) = 0.5$$

and:

$$\pi(\text{wait}|C) = 1$$

Remember that:

$$\pi(a|s)$$

means:

The probability that the agent chooses action  $a$ , given that it is currently in state  $s$ .

For example:

$$\pi(\text{move}|B) = 0.5$$

means:

When the robot is in state  $B$ , it chooses **move** with probability 0.5, or 50%.

Similarly:

$$\pi(\text{wait}|B) = 0.5$$

means that it chooses **wait** with probability 50%.

The probabilities of all possible actions in a state must sum to 1:

$$\sum_a \pi(a|s) = 1$$

Therefore, for state  $B$ :

$$\begin{aligned} \pi(\text{move}|B) + \pi(\text{wait}|B) \\ = 0.5 + 0.5 = 1 \end{aligned}$$

## 7. Transition Probability

The environment is described by:

$$p(s', r | s, a)$$

This means:

The probability of reaching next state  $s'$  and receiving reward  $r$ , given that the current state is  $s$  and action  $a$  was selected.

For example:

$$p(C, +10 | B, move) = 0.8$$

means:

If the robot is in  $B$  and chooses `move`, there is an 80% probability that it will reach  $C$  and receive  $+10$ .

Similarly:

$$p(B, -2 | B, move) = 0.2$$

means:

There is a 20% probability that the robot remains in  $B$  and receives  $-2$ .

## 8. Expected Immediate Reward

Now consider:

$$r(B, move)$$

The robot has two possible outcomes after choosing `move` in  $B$ :

| Outcome   | Probability | Reward |
|-----------|-------------|--------|
| Reach C   | 0.8         | +10    |
| Stay in B | 0.2         | -2     |

Therefore, the expected reward is:

$$\begin{aligned} r(B, move) &= 0.8(10) + 0.2(-2) \\ &= 8 - 0.4 \end{aligned}$$

$$\boxed{r(B, move) = 7.6}$$

This is an important distinction:

The actual reward is either +10 or -2, but the expected reward is 7.6.

## 9. Calculating a Return

Suppose the robot experiences this trajectory:

$$A \xrightarrow{\text{move}, +2} B$$

then:

$$B \xrightarrow{\text{move}, +10} C$$

The rewards are:

$$R_1 = 2$$

and:

$$R_2 = 10$$

Using:

$$G_0 = R_1 + \gamma R_2$$

we get:

$$G_0 = 2 + 0.9(10)$$

$$G_0 = 2 + 9$$

Therefore:

$$\boxed{G_0 = 11}$$

So the total discounted return for this particular trajectory is **11**.

## 10. Important Difference: Reward vs Return

These two concepts should not be confused.

### Reward

A reward is the immediate feedback from the environment:

$$R_{t+1}$$

For example:

$$R_{t+1} = +2$$

### Return

The return is the total discounted future reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

For our trajectory:

$$G_0 = 11$$

So:

Reward = immediate

while:

Return = discounted accumulation of future rewards

## 11. State-Value Function

The value of a state under policy  $\pi$  is:

$$v^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

In words:

$v^\pi(s)$  is the expected return when the agent starts in state  $s$  and follows policy  $\pi$ .

For example:

$$v^\pi(B)$$

asks:

If the robot is currently in state  $B$  and follows policy  $\pi$ , how much return can we expect in the future?

## 12. Bellman Equation for the State Value

The Bellman equation is:

$$v^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v^\pi(s')]$$

This equation can look complicated, but conceptually it says:

Value of current state = expected immediate reward + discounted value of next state

with an average over the actions selected by the policy.

## 13. Calculate $v^\pi(C)$

State  $C$  is the destination.

The robot can only wait:

$$\pi(wait|C) = 1$$

and:

$$P(C, 0|C, wait) = 1$$

Therefore:

$$v^\pi(C) = 0 + \gamma v^\pi(C)$$

So:

$$v^\pi(C) = 0.9v^\pi(C)$$

Thus:

$$0.1v^\pi(C) = 0$$

and:

$$v^\pi(C) = 0$$

This makes intuitive sense: once the robot has completed the delivery, there are no additional rewards.

## 14. Calculate $v^\pi(B)$

At  $B$ , the policy is:

$$\pi(move|B) = 0.5$$

and:

$$\pi(wait|B) = 0.5$$

Therefore:

$$v^\pi(B) = 0.5Q^\pi(B, move) + 0.5Q^\pi(B, wait)$$

Let's calculate the two action values.

## 14. Calculate $v^\pi(B)$

At  $B$ , the policy is:

$$\pi(move|B) = 0.5$$

and:

$$\pi(wait|B) = 0.5$$

Therefore:

$$v^\pi(B) = 0.5Q^\pi(B, move) + 0.5Q^\pi(B, wait)$$

Let's calculate the two action values.

### 14.1 Value of Move

If the robot chooses `move` :

- With probability 0.8, it reaches  $C$  and gets +10.
- With probability 0.2, it remains in  $B$  and gets  $-2$ .

Therefore:

$$q^\pi(B, move) = 0.8[10 + 0.9v^\pi(C)] + 0.2[-2 + 0.9v^\pi(B)]$$

Since:

$$v^\pi(C) = 0$$

we have:

$$\begin{aligned} q^\pi(B, move) &= 0.8(10) + 0.2[-2 + 0.9v^\pi(B)] \\ &= 8 - 0.4 + 0.18v^\pi(B) \end{aligned}$$

Therefore:

$$q^\pi(B, move) = 7.6 + 0.18v^\pi(B)$$

## 14.1 Value of Move

If the robot chooses `move` :

- With probability 0.8, it reaches  $C$  and gets +10.
- With probability 0.2, it remains in  $B$  and gets  $-2$ .

Therefore:

$$q^\pi(B, move) = 0.8[10 + 0.9v^\pi(C)] + 0.2[-2 + 0.9v^\pi(B)]$$

Since:

$$v^\pi(C) = 0$$

we have:

$$\begin{aligned} q^\pi(B, move) &= 0.8(10) + 0.2[-2 + 0.9v^\pi(B)] \\ &= 8 - 0.4 + 0.18v^\pi(B) \end{aligned}$$

Therefore:

$$q^\pi(B, move) = 7.6 + 0.18v^\pi(B)$$

## 14.2 Value of Wait

If the robot chooses `wait` :

$$P(B, 0|B, wait) = 1$$

Therefore:

$$q^\pi(B, wait) = 0 + 0.9v^\pi(B)$$

So:

$$q^\pi(B, wait) = 0.9v^\pi(B)$$

15. Solve for  $v^\pi(B)$ 

Recall:

$$v^\pi(B) = 0.5q^\pi(B, move) + 0.5q^\pi(B, wait)$$

Substitute the equations:

$$v^\pi(B) = 0.5[7.6 + 0.18v^\pi(B)] + 0.5[0.9v^\pi(B)]$$

Expand:

$$v^\pi(B) = 3.8 + 0.09v^\pi(B) + 0.45v^\pi(B)$$

Therefore:

$$v^\pi(B) = 3.8 + 0.54v^\pi(B)$$

Move the value term to the left:

$$0.46v^\pi(B) = 3.8$$

Therefore:

$$v^\pi(B) \approx 8.26$$

So, under this policy, the expected return from state  $B$  is approximately:

$$8.26$$

16. Calculate  $q^\pi(B, move)$ 

We previously obtained:

$$q^\pi(B, move) = 7.6 + 0.18v^\pi(B)$$

Using:

$$v^\pi(B) \approx 8.26$$

we get:

$$\begin{aligned} q^\pi(B, move) &= 7.6 + 0.18(8.26) \\ &\approx 7.6 + 1.487 \end{aligned}$$

Therefore:

$$q^\pi(B, move) \approx 9.09$$

This means:

If the robot is in state  $B$ , chooses `move` now, and then follows policy  $\pi$  afterward, its expected return is approximately 9.09.

### 17. Calculate $q^\pi(B, wait)$

We have:

$$q^\pi(B, wait) = 0.9v^\pi(B)$$

Therefore:

$$q^\pi(B, wait) = 0.9(8.26)$$

$$q^\pi(B, wait) \approx 7.43$$

So under this particular policy, the two action values are approximately:

$$q^\pi(B, move) \approx 9.09$$

and:

$$q^\pi(B, wait) \approx 7.43$$

These are **action values under policy  $\pi$** . They are not yet the optimal action values.

### 18. Optimal Value Function

The optimal state-value function is:

$$v^*(s) = \max_{\pi} v^\pi(s)$$

In words:

$v^*(s)$  is the maximum expected return that can be obtained from state  $s$  by following an optimal policy.

The Bellman optimality equation is:

$$v^*(s) = \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v^*(s')]$$

The important difference is:

**Under a specific policy:**

$$v^\pi(s)$$

we follow the probabilities specified by  $\pi$ .

**For the optimal value:**

$$v^*(s)$$

we choose the action with the highest expected value.

## 19. Optimal Value of State B

At state  $B$ , we have two actions.

### Move

$$q^*(B, move) = 0.8[10 + 0.9v^*(C)] + 0.2[-2 + 0.9v^*(B)]$$

Since:

$$v^*(C) = 0$$

we get:

$$q^*(B, move) = 7.6 + 0.18v^*(B)$$

### Wait

$$q^*(B, wait) = 0 + 0.9v^*(B)$$

Therefore:

$$v^*(B) = \max [7.6 + 0.18v^*(B), 0.9v^*(B)]$$

The optimality equation tells us to select whichever action has the greater expected long-term return.

|                                        |                                                  |
|----------------------------------------|--------------------------------------------------|
| State                                  | $A, B, C$                                        |
| Actions at B                           | move, wait                                       |
| Policy                                 | $\pi(move \mid B) = 0.5, \pi(wait \mid B) = 0.5$ |
| Discount                               | $\gamma = 0.9$                                   |
| $P(C, +10 \mid B, move)$               | 0.8                                              |
| $P(B, -2 \mid B, move)$                | 0.2                                              |
| Expected immediate reward of move at B | 7.6                                              |
| $v^\pi(C)$                             | 0                                                |
| $v^\pi(B)$                             | $\approx 8.26$                                   |
| $q^\pi(B, move)$                       | $\approx 9.09$                                   |
| $q^\pi(B, wait)$                       | $\approx 7.43$                                   |

# Question?

Thank you!